AALBORG UNIVERSITY
COPENHAGEN

Semester: CCT3

Title: AURORA search and recon rover

Project Period: 03-09-2025 -> 03-03-2026

Semester Theme: Development of embedded systems

Supervisor(s): Hossam Mohamed Farag

Project group no.: 7

Members:

Burhenne, Malte Bach

Lindau-Skands, Sebastian

Jensen, Lucas Ta*

Pages: 63

Finished: 02-03-2026

Aalborg University Copenhagen
A.C. Meyers Vænge 15 2450 København SV

Semester Coordinator: Hossam Mohamed Farag

Secretary: Nele Pernille Staun Hvid

Abstract:

This project researches the possibility of using autonomous search and recon rovers for rescue operations in hazardous environments, to minimise exposure to dangerous elements or conditions, by human staff, and thereby attempt to reduce the dangers and lifes lost associated with such fields of work. To aid with this we're exploring, among others, computer vision aided human detection, A* path finding, and autonomous driving

When uploading this document to Digital Exam each group member confirms that all have participated equally in the project work and that they collectively are responsible for the content of the project report. Furthermore each group member is liable for that there is no plagiarism in the report.

Table of contents

1 Introduction . . . 1
2 Problem Formulation . . . 1
2.1 Background . . . 1
2.2 Problem Analysis . . . 1
2.3 Problem Statement . . . 3
2.4 Objectives . . . 3
2.5 UN . . . 3
3 Methodology . . . 4
3.1 Research Methods . . . 4
3.2 Design and Implementation Methods . . . 6
3.3 Evaluation Methods . . . 8
4 State of Art . . . 9
4.1 Computer Vision . . . 9
4.2 Pytorch vs TensorFlow . . . 10
4.3 Robotics in Disaster Response . . . 11
4.4 JetBot . . . 12
4.5 path finding . . . 13
5 Requirement Analysis . . . 15
5.1 User Stories . . . 15
5.2 Use cases . . . 17
5.3 Flowchart of rover usage . . . 20
5.4 Requirements . . . 20
6 Implementation . . . 24
6.1 Software Implementation . . . 24
6.2 RPI4B . . . 50
6.3 jetbot . . . 50
7 Testing . . . 53
7.1 Testing Approach . . . 53
7.2 Preliminary Tests . . . 54
7.3 System Tests . . . 56
8 Discussion . . . 60
8.1 Jetbot setup . . . 60
8.2 Computer vision . . . 61
8.3 IMU . . . 62
9 Conclusion . . . 63
Bibliography . . . 65
10 Appendix . . . 67

1 Introduction

Search and rescue has always been a difficult task, and one that involves a lot of risk for the rescue personal involved. Due to the apparent dangers involved, we decided to create the aurora project, to facilitate safer search and rescue operations, by allowing an autonomous rover to assist the first responders in the work

The rovers functionality is based around an A* path finding algorithm along with a autonomous movement system, to enable both guided and automatic movement to the designated disaster area. A YOLO-based computer vision system was additionally implemented for aiding with the location of potential survivors on disaster struck areas

For communication between the various systems we divided it into an application layer and a physical layer. For the applications we decided to use websockets for the majority as it's a fast, easy and reliable continous data stream that multiple "clients" can be hooked into at once. The movement firmware however used http due to technical limitations

For the physical layer Nvidia had pre-implemented I2C based communication, and we therefore had to do little in this area.

2 Problem Formulation

2.1 Background

Natural and man-made disaster continue to pose growing challenges to emergency response teams worldwide. Climate change is intensifying the frequency and severity of extreme weather with 838 emergency responders dying from fatal injuries during disaster response operations in 2015 [1]. These statistics highlight the need for technological solutions that reduce human risk while improving response effectiveness. This need positions robotics as a useful technology to be used in disaster response.

Robotics has already been used numerous times in the past in the context of disaster response. The first instance was during the events of 9/11, where on September 11, 2001, where the Center for Robot-Assisted Search and Rescue (CRASAR) deployed small mobile robots to investigate the rubble of the World Trade Center. [2]

Later, following Hurricane Katrina in 2005, the first deployment of unmanned aerial vehicles (UAVs) occurred. [3] They were primarily used for emergency structural inspections. Similarly, UAVs were used for determining radiation levels and inspect damage to the reactors in 2011 following the Fukushima nuclear emergency [3]

2.2 Problem Analysis

First responders are exposed to a plethra of high risk hazards, from raging fires to biohazardous material, and occasionally even radiation. During search and rescue operation it's unfortunately a rather common sight to loose a first responder or two due to the inherrent risks tied to the job[4]

Hazards

First responders often have to deal with precarious conditions when handling both natural and man-made disasters. This poses significant risk to disaster response personal, who may have to spend extended time in these hostile environments. Some of the most hazardous places includes areas with toxic gasses, chemical contamination, radiation, and unstable structures.

Toxic gas exposure is one of the most immediate and lethal hazard in disaster environments. An example of such gas is Hydrogen sulfide, which is commonly found in sewage systems, industrial accidents and confined spaces [5]. The gas is the second most common cause of fatal gas exposure. It's highly toxic, and simply inhaling it can lead to a lethal dose in minutes. What makes it especially dangerous is even at lower exposure times it can prove fatal, as concentrations above 1000 ppm cause immediate unconsciousness and death within just minutes. Assuming the affected person doesn't have assistance, the immediate unconsciousness is a death sentence in itself, as they won't be able to remove themself from the lethal environment. Hydrogen sulfide does however have an odor of rotten eggs, which in theory is an effective warning sign and should make it easier for someone to realize that they are in contact with the gas. Unfortunately, it also has the side effect of olfactory paralysis, which is the loss of smell. This quickly renders the crucial warning sign unreliable, and may lead to a false sense of safety as the affected person might think they are clear of the gas, when in reality it has simply paralyzed their sense of smell. Hydrogen sulfide is also proven to be a notable hazard to rescuers as they often get affected themselves in their effort to save people from affected environment. Reports suggest that up to 25% of hydrogen sulfide toxicity fatalities involve rescuers [5].

Another dangerous scenario is chemical contamination. Chemical disasters create large zones in the surrounding areas where entering is highly toxic and outright lethal in some cases. The 1984 Bhopal Disaster was one of those cases. As one of the worst chemical accidents in history, it is estimated that up to 5000 people died in the first 2 days following exposure to chemicals [6]. It is also estimated that the total death toll might be as high as 20,000. These environments causes significant challenges for disaster response teams, as entering is impossible without extensive protective equipment and decontamination efforts.

Unstable structures caused by the collapse of buildings and other structures are a common occurrence on incident scenes. First responders may risk secondary collapses and entrapment when navigating these, which can lead to injury and in some cases even death [7]. Already collapsed structures or parts can additionally have consequences for the rescue teams which could make recovery slow or even unfeasible. Hazardous objects like sharp objects as glass and debris might be spread throughout the area, and it can also cause the aforementioned hazard types as gas and chemicals to get exposed from the broken structures if eg. pipes or similiar where present. In addition to increased difficulty in traversing these environments, there is also a constant threat of further secondary collapses, where weakened building elements can collapse without warning. This makes human reconnaissance extremely hazardous even for trained rescue personnel.

These hazardous conditions, among many others not mentioned here, showcase the critical need for remote reconnaissance capabilities that can assess disaster environments without risking the human lives of disaster response personal. Traditional assessment involving

humans physically present are dangerous and in some cases also impossible in the immediate aftermath of disasters, which is exactly when swift reconnasence and rescue operations are the most critical.

Robotics in disaster response

To ease the workload, human first responders have to do directly inside these dangerous areas, robots can be used for tasks such as reconnaissance. This ensures that disaster response teams can already have intel on an area before sending in humans, which may lessen the time they have to spend there. Robots however also have problems that needs to be addressed.

Robots have a plethera of limitations in terms of range, battery capactiy, manouverability, and many more. one of the greatest issues with robots in reconnasence, and for that matter any piece of critical infrastructure or operations, is the reliability of the software and the communication protocols, something that we hope to be able to address during the course of this project

2.3 Problem Statement

How can we design a remotely controlled, recon rover suitable for assisting first responders in gathering intelligence, and detecting human suvivors

2.4 Objectives

- How do we design a system for the rover to assist with finding humans?
- How do we design a system for the rover to assist with navigation?
- How do we design a system for the First responders to interface with the rover?

2.5 UN

This project seeks to contribute to the following UN's world goals:

- 3: Good health and welbeing. The primary purpose of the recon rover is to detect human lifesigns, as to help the first responders create an efficient battleplan, whilst minimizing their exposure to hazardous environments.
- 9: Industry, innovation & infrastructure. By facilitating the rapid recovery from catastrophes, we are improving infrastructure, and allowing for repurposing of wasteland as industry space.
- 15: Life on land. By facilitating the rapid cleanup of disaster areas, we are contributing to the restoration of terrestrial areas, and prevention of biodiversity loss.

3 Methodology

This chapter describes the methods and approaches used for developing the search and recon rover and answer the problem formulation presented in Section 2. The chapter is structured chronologically, following the development phases of the project: Research, design, implementation and evaluation.

3.1 Research Methods

The research methods focused on understanding the problem the project strives to solve. Furthermore it also included examining possible technologies relevant to the implementation of it, and already existing disaster response robots. This section focuses on how literature was reviewed and chosen, how existing solutions were analyzed and how technologies were compared.

Literature Search and review

A fundamental part of the research process was the search and review of relevant literature. This was the source of any knowledge regarding the problem, existing solutions and possible technologies, and thereby provided a basis upon which the rest of the project could stand. Some key focus areas was to understand the problems faced in disaster response and the applications of robotics. Other important areas was computer vision, path finding algorithms, and what hardware to use for the actual rover.

The review was primarily conducted based on academic textbooks, technical documentation and research papers. Academic sources were identified primarily through Primo, Google Scholar and IEEE Xplore. Other types of sources such as technical documentation was found through general search engines.

How the sources were picked were dependent on what they were needed for. In the beginning of the project, multiple sources were found related to the problem. This enabled the creation of a fundamental knowledge base, from which the problem could properly be analyzed. In the outset of this process, general sources related to robotics in disaster response were found. From there it could be determined what problems needed further examination, for instance in regards to which disaster types robotics would be most beneficial for. Furthermore, all sources had their relevance, quality, and timeliness determined, in order to get the most optimal sources. In other parts of the project, sources were also found based on when the need for them arose in the specific situation.

This approach ensured that the understanding of the problem and the design of the rover was comprehensive and informed by credible sources.

Existing Solution Analysis

Other examples of developed robots used for disaster responses were analyzed to identify common design patterns and proven features. One of the things this analysis revealed was that these solutions typically included remote video streaming for situational awareness. Besides that, common challenges and problems were also examined to see if any of them could be addressed. In a matter of fact, situational awareness itself was generally a common problem discussed, and something which could be improved upon. Lastly, common use cases for robots, specifically ground vehicles, were determined. By understanding their role in disaster response, more educated decisions could be made when it came to the design and what to prioritize. These findings directly contributed to the decisions made in the requirement analysis (Section 5).

Problem Domain Research

To better understand the context of where the rover would have to operate, multiple different types of disaster scenarios and their challenges were researched. As earlier mentioned, the found use cases robotics in disaster response were used as a starting point for this process. Specific disaster scenarios were picked based on how deadly and unpredictable they were, as these would be the situations where robotics would be most beneficial. The research in this area solidified the need for autonomous assistance to protect first responders, and proved that the rover would in fact benefit disaster response.

Technology Evaluation

Technologies which were deemed potentially useful for the development of the rover was also researched. Most importantly, technologies with the same overall purpose were compared in order to find the one most optimal for the task. The criteria from which they were compared were dependent on the exact technologies and the needs they would have to fulfill. There were however some common themes such as hardware compatibility, processing capabilities, development ecosystem, efficiency and speed.

3.2 Design and Implementation Methods

Based on the findings from the research, the actual rover could be designed and implemented. During the process of going from a problem to an actual product, multiple methods were used.

Requirements

Requirements that the rover would have to fulfill were found using a user centered approach which progressed through four stages: stakeholder identification, user story creation, use case analysis and MoSCoW prioritization. This approach ensured that the system design remained grounded in real world needs, while also making the scope more manageable and objective.

The process began with stakeholder identification, where two key groups were chosen: Emergency response teams (the primary operators of the robot) and incident commanders (higher-up decision makers). By identifying these stakeholders, the requirements gathering process could capture diverse perspectives from all the users of the rover, and ensure that the rover address all of their needs. This aligns with a user-centered approach, as it prioritizes understanding the users needs before the technical implementation begins.

Multiple user stories were then created from the perspective of each stakeholder group, following the format "As a [role] i want to [action] so that [reason]". This format was chosen as it focuses on the users needs and values, rather than technical specifications, keeping the design aligned with actual use cases for the time being. For example, rather than just deciding "add video streaming", the user story underlines the underlying need: "As a rescuer I want to be able to receive video footage from the robot, while maintaining a safe distance from any hazards, so that I can control the robot over longer distances from safety". This difference ensures that the technical solutions address the fundamental problem (maintaining safety) rather than just implementing features that feel right.

Following this, these user stories were then transformed into use cases that described specific system interactions. Use cases were used to translate abstract and ambiguous user needs into concrete functional scenarios, making them more actionable for design. For instance the earlier mentioned user story about video streaming was turned into the use case "Receive video footage", a much more precise and concise version of what the rover needs to be able to do. Furthermore the use cases were also used to create a use-case diagram used to map out all the use cases, as well as interactions and dependencies between them, primarily to be used in the next and last step.

From the use cases, concrete functional and non-functional requirements were extracted. Functional requirements specify the system capabilities, or what it should be able to do. Non-functional requirements define quality attributes, or how well the system should perform in certain areas. These requirements were then categorized using the MoSCoW model (Must have, Should have, Could have, Won't have). Must-Have requirements defined the minimum viable product, and included requirements like video streaming and remote control. Should-Have features like computer vision object detection were important, but not necessarily vital for the project. Could-Have features were implemented only if time permitted. The MoSCoW model made it possible to manage the scope of the project throughout the development. By first implementing the requirements needed for the product to work, the risk of running out of time with a product that didn't fundamentally work was significantly smaller. For example

having a rover capable of pathfinding would be irrelevant if the rover wasn't able to move in the first place.

The entire process of engineering requirements, from stake holders and user stories to concrete requirements and their prioritization, created a clear roadmap connecting stakeholder needs to the technical implementation. This user-centered approach ensured that the entire development aligned with the goals of the project rather than being driven by technical curiosity alone.

Iterative Development

An iterative development approach was used throughout the project. Rather than attempting to plan a complete system up front and follow a rigid plan without flexibility, the system was developed using iterative cycles. This better reflects the inherent uncertainty and unpredictability that comes with software development. Each iteration involved designing, implementing and testing a functional component of the system, from which future iterations could be planned based on the result of that.

This iterative approach proved especially useful when dealing with embedded systems. Hardware components are significantly more unpredictable than software components, and integration issues often aren't visible until hardware and software interact with each other. During the process of integrating the software with the Jetbot, numerous problems were encountered, requiring multiple iterations of software component to make them work efficiently together.

The software architecture also evolved iteratively. The camera frame capture on the backend is an example of this. Early versions relied on the questionable approach of spawning ffmpeg as a subprocess to capture MJPEG frames from the camera. It then piped the raw MJPEG stream to stdout, then manually parsed the byte stream to identify individual JPEG frames. While it was functional, this approach was overly complex for the task it needed to do, and was also inefficient due to significant overhead. Due to this, a new iteration of the video capture was created using OpenCV instead. This approach directly accesses the camera without spawning external processes, and by that eliminating subprocess overhead. Besides that, it also resulted in much cleaner and more concise code, making future iterations of the same system easier to develop.

This iterative methodology allowed the project to adapt to real-world constraints and continuously improve, validating the decision to prioritize flexibility over rigid planning.

Task Management and Workflow

Task management was organized using a kanban board (Figure 1) with four columns: To Do, In Progress, Review and Done. This visual workflow was chosen to complement the iterative development process, to improve communication and provide an overview for the team.

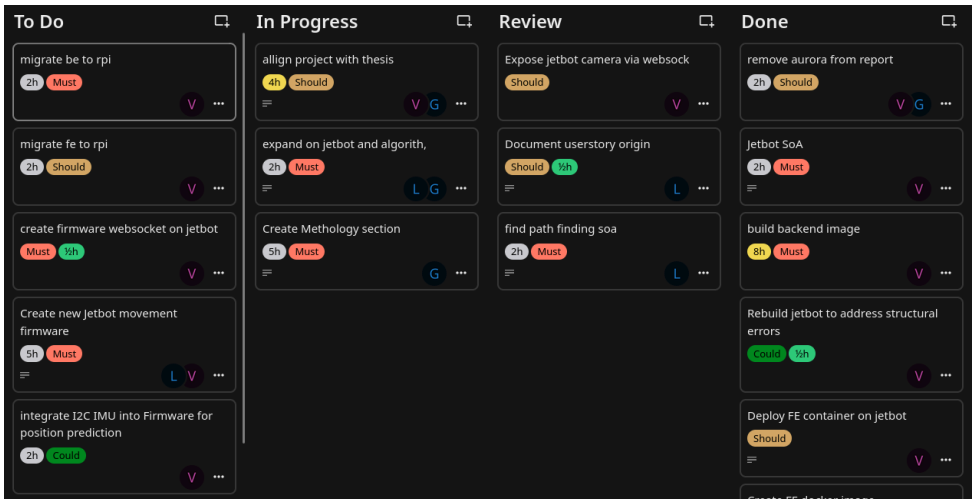


Figure 1: Kanban board workflow showing task organization

The tasks covered various activities such as implementation work, report writing and research. Significantly complex tasks were also broken up into smaller more manageable tasks to enable parallel work and incremental progress. Each task was labeled with MoSCoW priority the same way as the requirements from earlier, again representing how important they were. Furthermore tasks were given an approximate time estimate for supporting workload distribution and time management. The review column served as a quality checkpoint. Completed tasks required peer validation before it was marked as done.

This workflow supported the iterative methodology by providing continues visibility into the the work status and how much work remained. It also provided better transparency as it was easy to figure out who was responsible for specific tasks, and what the different team members was currently working on.

3.3 Evaluation Methods

We came up with a hypothesis for the experiment prior to execution that we would then evaluate the results against to determine wether or not the rover meets our requirements. If results deviated significantly from our hypothesis we would reitterate over the implementation till the results where close enough to be suffice for a Proof of Concept

4 State of Art

4.1 Computer Vision

The book "3-D Computer Vision" by Yu-Jin Zhang goes into depth with what computer vision is, what algorithms are most commonly used, as well as how to implement these and their potential uses in robotics. [8]

Computer vision is a discipline in AI and computation that aims to emulate the functions of the human vision system (HVS). The HVS is a complex process that begins with the sensation of light in the cone receptors of the eye and continues through perception and interpretation in the brain, where multiple neurological "algorithms" work with memory and prior knowledge to interpret signals and determine what we should be aware of, such as a hot plate or a sharp knife.

Computer vision is artificially made HVS, and therefore needs everything that HVS needs in order to be successful. This includes at least one, but preferably two cameras for depth vision ,a memory bank it can pull knowledge from, as well as some processing power that interprets it all.

In the following sections this article will make a deep-dive in how computer vision works at a fundamental level, and how this could be beneficial for robotics.

The relationship between human vision and computer vision

The human visual system is a complex interaction between the eyes and the brain, where photic information is interpreted into shapes, colors, and movements. Computer vision seeks to mimic this process through a technical pipeline consisting of:

- Image capture – cameras and sensors register light and color.
- Pre-processing – noise reduction, contrast enhancement, and filtering.
- Feature extraction – identification of edges, corners, colors, and textures.
- Interpretation and classification – recognition of objects and relationships.

Whereas human vision combines physical and psychological processing, computer vision uses mathematical models, such as lens and image formation models, to describe the relationship between reality and the recorded image.

Key areas in computer vision

According to Zhang's book, computer vision consists of several key areas:

- Camera calibration – determining the camera's internal (focus, lens distortion) and external (position and orientation) parameters.
- 3D image formation – constructing three-dimensional models from two or more image systems.
- Image analysis and segmentation – dividing an image into meaningful parts (objects, background, etc.).
- Object recognition and classification – identifying what is seen.
- Motion analysis – measuring and interpreting movements in an image sequence.

4.2 Pytorch vs TensorFlow

Pytorch

Pytorch is a machine learning library with focused development for the programming language python, although it also does have c++ and java branches. [9] Pytorch has a large and robust ecosystem with many packages that can be added upon the pytorch installation for ease of development in diffent areas, for example flair; natural language processing package which has a focus on biomedical texts[10]. Pytorch is for example used in the industry by big players such as stanford university and amazon advertising, just to mention a few.

Pythorch has a robust CUDA integration which helps tremendously with traning and using your AI models, since it fascilitates thew utilisation of your GPU's CUDA cores, who'm are capable of trillions of operations per second, thereby significantly increasing speeds

Pytorch has many partners in the cloud computing Industry therefore there is a massive emphasiz on cloud computing on their website [9]

TensorFlow

Tensorflow is a python focused machine learning library from google, suitable by extensions for various tasks, such as reinforcement learning via keeras. It was developed by google to address many of the issues in alreaady existing infrastructure, and provide an open source platform to develop larger AI models. [11]

Tensorflow is among the industry standards when it comes to AI development thanks to it's large eco system, and ability to be delopyed on web, desktop applications, IoT devices, and more. [11]

Additionally, tensorflows strong integration with Nvidia and the whole CUDA platform makes it attractive for many machine learning developers, as this can signifficantly increase training rates, and thereby speed up development

as shown in [12] the performance can be great, especially when paired with a cuda eneabled GPU, which of course is promising for a project such as ours

Comparison

Based on the shallow learning curve, and the ease of debugging of pytorch, along with the more optimal ressource management on lower end hardware [13] Pytorch was the obvious choice for us. Whilst tensorflow would've provided many nice to have features, such as baked in standards, and reliance on static graph concepts [13] Ultimately the time and ressources mattered more

4.3 Robotics in Disaster Response

The use of robotics has already now significantly improved the effectivity of disaster response. [14]. Especially in search and rescue operations, where they are primarily used as alternatives to human personnel in hazardous areas, such as those with significant smoke, damaged infrastructure, or inaccessible areas. Robots make it possible for disaster response to cover areas that would otherwise be too risky for humans to enter, and locations that humans would be unable to access.

Currently, numerous operations have utilized robots for disaster response [14]. One of the simpler applications for these robots has been to serve as "eyes and ears" for response teams, enabling them to collect video and audio information from hazardous areas. However, they can also be utilized for more sophisticated tasks. An example of these occurred during a residential building fire in Essen in February 2022, where robots, in addition to collecting videos, where also utilized to create 3D models of the environment using a LiDAR system [15]

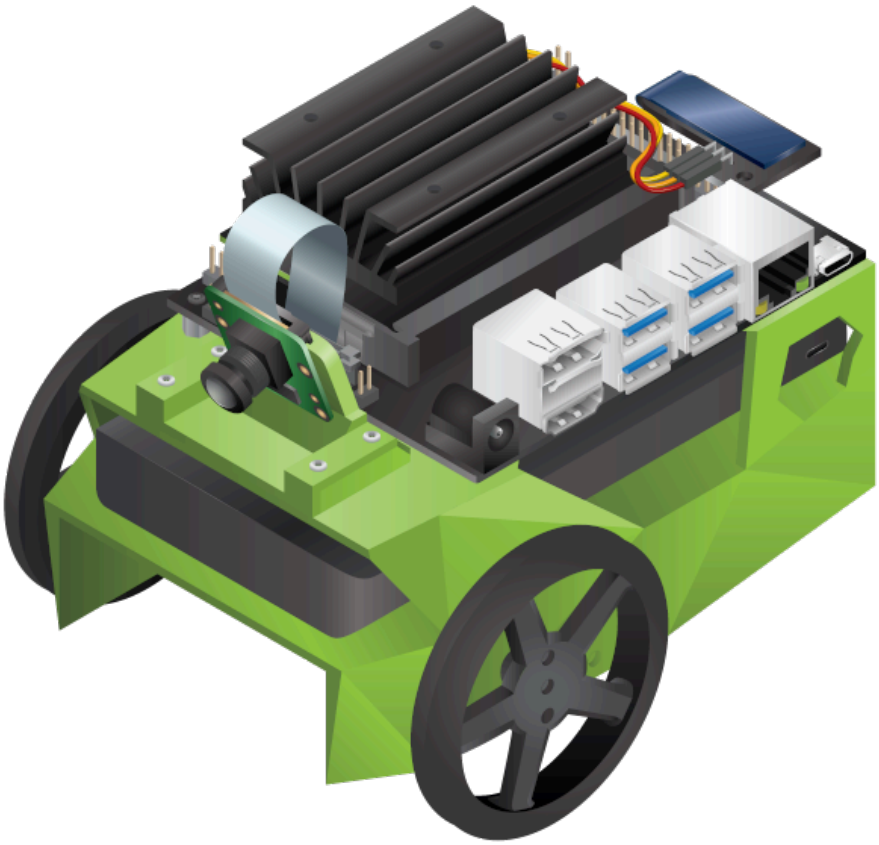
The two main categories of unmanned robots used in disaster response are Unmanned Ground Vehicles (UGVs) and Unmanned Aerial Vehicles (UAVs). Both have distinct use cases where they offer advantages over the other. UAVs posses superior mobility and the ability to fly, making them more suitable for reconnaissance, especially in scenarios involving collapsed infrastructure. UGVs on the other hand, are generally better equipped for investigating indoor areas, as they can more easily perform actions such as opening doors or moving obstacles [14]. Similarly they are also better equipped to investigate areas up close, as they are usually more sturdy and stable than their aerial counterparts. Furthermore, UAVs also carry as risk of spreading harmful gases and other airborne hazards, something UGVs generally avoid due to rarely being equipped with rotors.

Within UGVs, various design approaches exist. For instance, robots can be outfitted with wide array of equipment depending on their purpose, including multiple cameras, microphones, GPS, and various sensors. Many also incorporate built-in processing capabilities, such as odometry algorithms, environmental mapping, object detecting, and route planning.

However, the usage of robots in hostile environments doesn't come without challenges. One of the most significant obstacles observed is communication difficulties. As infrastructure is often damages during disasters, internet-based communication is frequently unreliable, if not entirely impossible [14]. Direct radio communication can also be challenging if robots are operating indoors, as walls and other objects can weaken the signal [15]. This approach also has limited range. Situational awareness is also a problem which is frequently discussed. The majority of robots has some kind of camera which offers the operator a view from the rover, but further assistance such as data visualizations and maps can also be useful. Other challenges include technical obstacles such as reliability and flexibility. [16]

4.4 JetBot

The Jetbot is a small IoT robotics development kit made by Nvidia, to encourage the younger generation to get into robotics, and engineering in general.



The hardware architecture is based on the jetson nano, which is an IoT based development kit by Nvidia designed for AI. It combines a multicore CPU with an integrated GPU and 4GB of ram, to excel at basic compute tasks. The jetbot's software eco system is built upon python3.8 and jupyter, along with pytorch 1.x and tensorflow. The suite provides a pleathera of examples both for movement and for computer vision using the inbuilt camera. [16]

Jetbot is pitched primarily as an educational tool meant for highschoolers and undergrads to introduce them to AI, IoT and robotics. Additionally jetbot has seen adoptation in certain research environments, where, due to the open linux nature of the project, it can easily be adapted with the Robot Operating System or ROS

Compared with other similiar platforms, jetbot's key strentghts lie in it's industrial grade AI support, along with the open source software environemtn, and community wide support and development of different projects. [16]

Despite this the jetbot does present a substantial amount of technical limitations in 2026, such as 8 year outdated operating systems, non standard partitioning schemes, and a heavy reliance on cloud technology.

4.5 path finding

When referring to path finding there are two algorithms that are primarily discussed. These are the Dijkstra and A* path finding algorithms [17]

Dijkstra

Dijkstra is one of the most commonly used pathfinding algorithms. It operates on the traversal space, for example a weighted graph or grid, and examines all the nodes to determine shortest path. Because it explores the entire traversal space until it reaches the end point and then returns the shortest path, it can be a time consuming and inefficient algorithm for use-cases where speed is essential. [18]

As Dijkstra originally described algorithm: "We consider n points (nodes), some or all pairs of which are connected by a branch; the length of each branch is given. We restrict ourselves to the case where at least one path exists between any two nodes. We now consider two problems." [19]

- Problem 1: Construct the tree of minimum total length between the n nodes.
- Problem 2: Find the path of minimum total length between two given nodes.

given how Dijkstra is constructed, it will be fast to find the shortest path between any combination of nodes once the algorithm has traversed all the nodes. However, because it evaluates all combinations it can be slow and has a suboptimal time complexity expressed as $O(n^2 + m)$[18]. Because of the high time complexity, it is not recommended to use this algorithm on a large grid, and is not applicable for use in this project due to the need for semi-dynamic mapping.

A*

A* is one of the most commonly used pathfinding algorithms. It works by viewing the traversal space as a grid of nodes that have three values [20].

- g(n) - The exact cost so far, or distance from the starting point
- h(n) - The heuristic, the distance from the end point
- f(n) - The combined estimate used to prioritize nodes. $f(n) = g(n) + h(n)$[21]

Since A* views these values as costs, it will select the cheapest possible path, or move to the cheapest possible node on the grid. This focused search makes it a faster and more controllable pathfinding algorithm, since it is possible to change the weight of the different values.

The A* algorithm normally prefers the route with the lowest h(n) when two paths have the same cost, however our obj(n) influences it to select the path requiring the least time when two paths have the same cost. This behavior is ideal for search and reconnaissance as every second counts, and a few extra meters can be the difference between life and death. Furthermore, because it doesn't examine all of the nodes and performs are more focused search, it is substantially more time- and computationally efficient then Dijkstra.

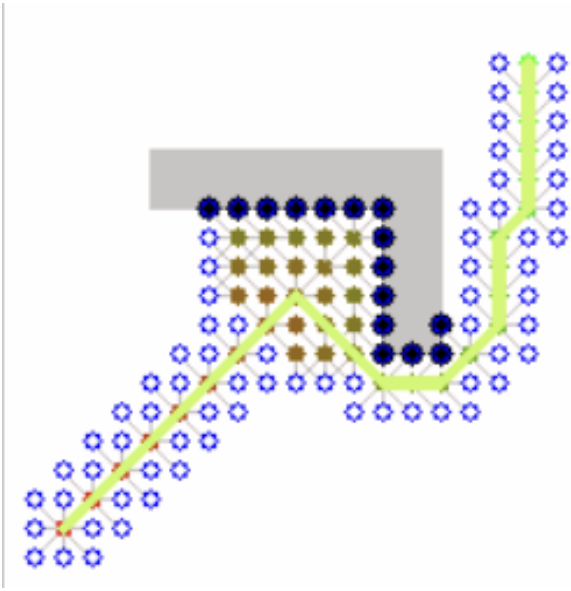


Figure 2: finised astar search visulisation

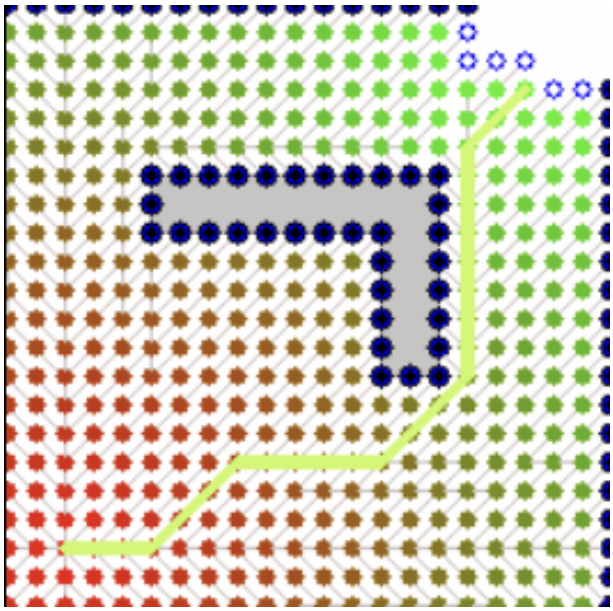


Figure 3: finised dijksra search visulisation

5 Requirement Analysis

Following the problem formulation and general objectives established in Section 2, this chapter presents an analysis of the system requirements for the project.

5.1 User Stories

To get a better overview of the use cases and requirements for the robot, several user stories has been created. These user stories are from the perspective of different groups of end-users, who are assumed to be key stakeholders. Each user story follows the format of describing something the user wants to achieve, as well as why they want to achieve this. All needs may not be addressed in the final project due to time constraints and other limitations, but the stories still give a point of reference to the what should be prioritized. Each user story has been given a specific ID for easy reference in later chapters.

To determine what stakeholders should be included in the analysis, we analyzed a large number of emergency reports and disaster videos. One of the main findings were the two main categories of people present: the emergency response team often compromised of police, fire department personnel, emergency medical services, and the incident commanders. Because the incident commander has a different role compared with the others, we treated that position as a separate entity from the emergency response team. [22]

Emergency Repones Team

ID	User Story
US-ERT-01	As an emergency response team member, I want to control the rover's movement direction so that I can navigate it through the disaster environment.
US-ERT-02	As an emergency response team member, I want to view a live video feed from the rover so that I can observe and navigate the environment and locate survivors.
US-ERT-03	As an emergency response team member, I want to enable AI-powered human detection so that I can be assisted in spotting survivors I might miss.
US-ERT-04	As an emergency response team member, I want to view the path the rover has planned on a map, so that I can steer the rover in a safe fasion towards it's destination.
US-ERT-05	As an emergency response team member, I want to access the control interface from any web browser, so that I can quickly operate the rover without installing special software.
US-ERT-06	As an emergency response team member, I want to see the current status of the rover (an example could be if human detecting is enabled) so that I can confirm that my commands are executed correctly.
US-ERT-07	As an emergency response team member, I want to operate the rover and see the video feed even if the human detection fails to work so that I can still perform manual reconnaissance.
US-ERT-08	As an emergency response team member, I want to see the rover's estimated position on the map, so that I can track coverage progress, and get an idea of where it is.
US-ERT-09	As an emergency response team member, I want to be able to lock controls so that I can avoid accidental movement in certain situations.
US-ERT-10	As an emergency response team member, I want to log detected survivors and their location so that they can be found later.
US-ERT-11	As a rescuer I want the robot to automatically navigate to the edge of the disaster area and let me take over from there so that I can focus on other tasks in the meantime.
US-ERT-12	As a rescuer I want to be able to automatically recall the robot and have it attempt to safely return by itself, so that so that I can retrieve the rover without manual navigation when the mission is complete

Table 1: Emergency Response Team User Stories

Incident Commander

ID	User Story
US-IC-01	As an Incident Commander, I want to view the rover's search coverage on a map so that I can coordinate search operations.
US-IC-02	As an Incident Commander, I want to receive information about found survivors so that I can dispatch ground teams to these locations.
US-IC-03	As an Incident Commander, I want to monitor the operational status of the rover remotely so that I know if the system is functioning correctly.
US-IC-04	As an Incident Commander, I want to view detection logs of found survivors so that I can plan accordingly.

Table 2: Incident Commander User Stories

5.2 Use cases

Based on the user stories from the previous section, a set of use cases has been created. Use cases act as more concrete functional scenarios and describe the interactions between actors and the system to achieve specific goals. Similar to the user story, each use case has been given a unique ID, following the format `UC-xx`. The use cases are formated as a table containing information about the use case. This information includes the ID, the name, the primary actor, description, preconditions and the related user stories. It is important to note that while the primary actor might often correlate with the stakeholder, this is not always the case. The primary actor is the actor who initiates the function, while the stakeholder is someone with a stake or interest in it. A precondition is a condition that must be met before the use case can be executed. Preconditions such as the rover being operational and the operator being connected to the interface are basic system assumptions and are considered implicit. Due to this they will not be explicitly mentioned in the table.

ID	Name	Primary Actor	Description	Precondition(s)	Related Userstory
UC-01	Control Rover Movement	Operator	Operator sends directional commands to navigate the rover through the disaster enviorment.	-	US-ERT-01
UC-02	View Live Video Feed	Operator	Operator view live video stream from the rover's camera to observe the environment and identify survivors.	-	US-ERT-02
UC-03	Toggle Human Detection	Operator	Operator enables or disables human detection on the video feed.	-	US-ERT-03
UC-04	View Planned Path	Operator	Operator views the rover's calculated path on the rover interface.	Destination configured, path calculated	US-ERT-04
UC-05	Connect to Control Interface	Operator	Operator accesses the rover control system through a web browser	-	US-ERT-05
UC-06	Monitor Rover Status	Operator, Incident Commander	User views the rover's current operational status. Examples could be current speed and if human detecting is enabled.	-	US-ERT-08, US-IC-01
UC-07	View Rover Position	Operator	Operator views the rover's estimated position on the map	Position tracking active	US-ERT-08
UC-08	Lock/Unlock Controls	Operator	Operator locks or unlocks movement controls to prevent accidental input in certain suturations.	-	US-ERT-09
UC-09	Detect Human	System	System automatically detects humans in the video feed using AI-based detection.	Human detection enabled	US-ERT-03
UC-10	Log and Report Survivor Detection	Operator	Operator logs detected survivors to avoid false positives. System automatically captures location and timestamp.	-	US-ERT-10, US-IC-02, US-IC-04
UC-11	View Rover Coverage	Incident Commander	Incident commander views search coverage area on the map.	Position tracking active	US-IC-01
UC-12	Autonomous Navigation to Disaster Area	System	Rover automatically navigates to the edge of the disaster area before transferring control to the operator.	Destination Configured	US-ERT-11
UC-13	Autonomous Recall	System	Rover automatically attempts to return to its starting position when given the command to do so.	Recall command given	US-ERT-12

Table 3: Use Cases

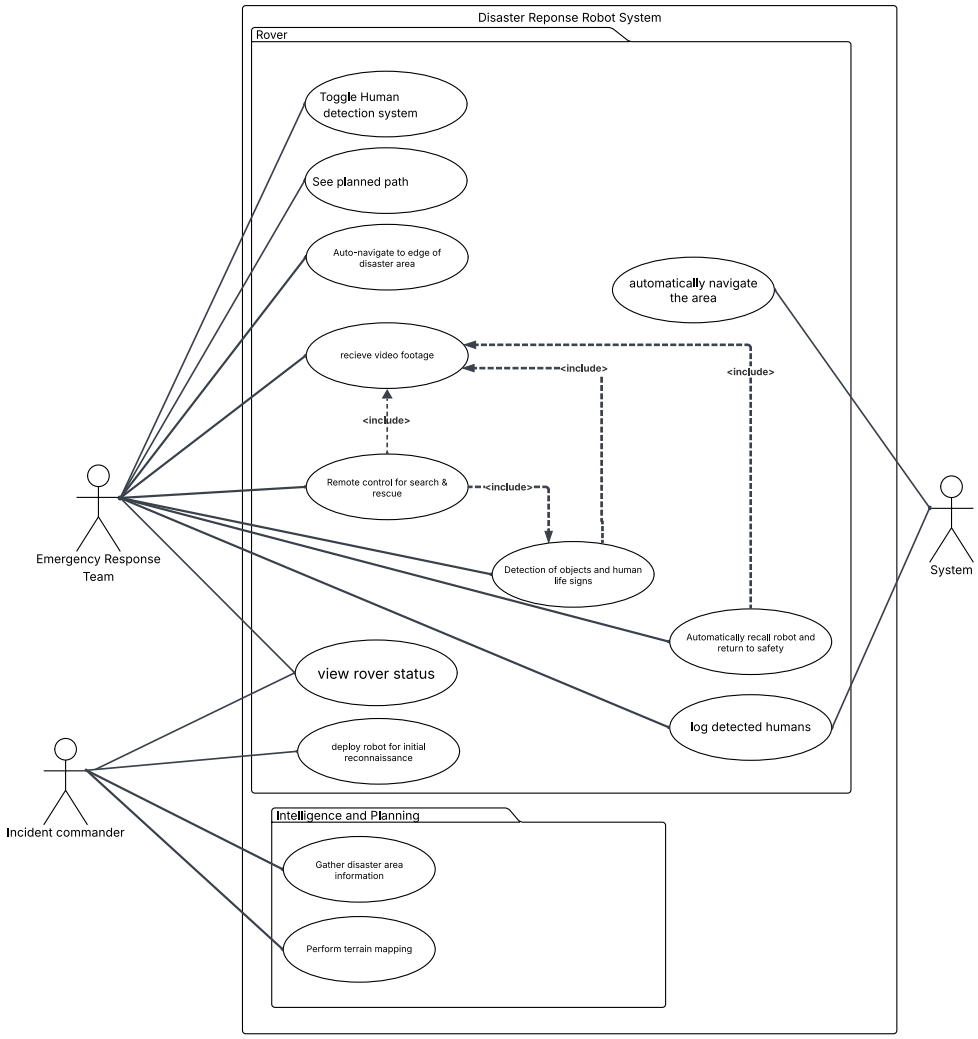


Figure 4: usecase diagram

5.3 Flowchart of rover usage

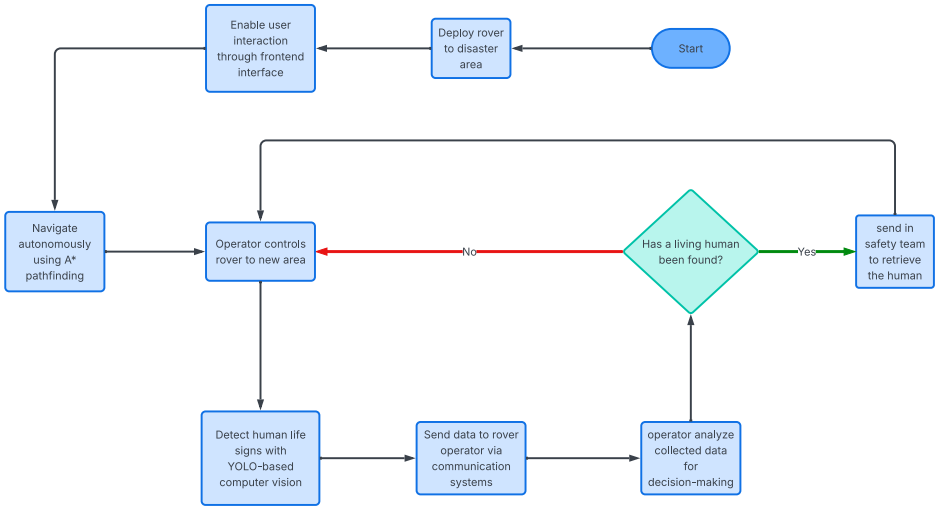


Figure 5: flowchart of the deployment process

Above on Figure 5, the flowchart of the rover usage process can be seen. The process starts by deploying the rover and enabling the user interaction. The rover then moves to the site of the disaster and then moves around searching for signs of human life. If a survivor is detected the operators are notified of such and safety personel is send in and the process repeats until it is determined that there are no more survivors.

5.4 Requirements

Based on our user stories and research, we have created a list of the final requirements that the rover should fulfil, and which the later system design and implementation will be based off. These are categorized using the MoSCoW prioritization method, which classifies requirements based on their importance for project success. This allows the implantation of the most vital functions first, and then focusing on more complementary features after the rover is in a functional state.

MoSCoW is a prioritization method that categorizes requirements in four categories.

- **Must Have**: Critical requirements without which the project fails. Without these the project fails to meet the core objectives. This also represents the minimum viable product, and the first iteration of the prototype.
- **Should Have**: Important features that add significant value, but aren't vital for initial prototype. They should be included if time and resources permit it. Excluding these would impact functionality, but not prevent the system from working.
- **Could Have**: Desirable enhancements that can be omitted without major impact. They can be described as "nice-to-have" features that improve the functionality or improve user experience, but can be excluded without major consequences.
- **Won't Have**: Features that has been determined to be out of scope for the project. These features are not included in the report.

Furthermore each requirement is also classified as either functional or non-functional. Functional requirements can be described as what the system does, while non-functional requirements is how well it performs.

Functional Requirements

Must Have

ID	Requirement	Justification	Related
FR-01	The system must allow the operator to control rover movement in multiple directions	Core functionality, without movement control, the rover cannot be operated at all.	UC-01
FR-02	The system must stream live video from the rover camera to the operator interface	Core functionality, operator cannot navigate or search without any visuals.	UC-02
FR-03	The system must provide remote access to the control interface.	Core functionality, the rover has to be operated remotely in order to keep human responders out of hazardous environments.	UC-05

Table 4: Must have functional requirements

Should Have

ID	Requirement	Justification	Related
FR-04	The system should identify humans and visually indicate them on the video feed.	Key feature that makes it a specialized reconnaissance robot, but manual identification of humans is technically a viable fallback.	UC-09
FR-05	The system should support toggling automatic human detection	Natural complement to FR-04, the operator should be able to disable human detection when not needed to improve system performance.	UC-03
FR-06	The system should display the rover's estimated position on a map.	Significantly aids coordination and coverage tracking, but operator can complete their work without it.	UC-07
FR-08	The system should display current rover operational status to the operator.	Important for confirming commands executed correctly, and to quickly get an overview of the rovers status.	UC-06
FR-09	The system should have core functionality still operating if optional features fail.	Reliability is important in disaster scenarios. By implementing graceful deration, complete failure can be prevented even if parts of the system stops working.	UC-03
FR-10	The system should allow the operator to lock movement controls.	Safety feature to prevent accidental input. Important but not essential.	UC-08

Table 5: Should have functional requirements

Could Have

ID	Requirement	Justification	Related
FR-11	The system could calculate navigation paths to specified destinations.	Useful for planning and necessary for possible autonomous features, but manual control is the primary mode of the rover.	UC-04
FR-12	The system could visualize the planned path on the interface.	Depends on FR-11, only possible if path calculation actually exists.	UC-04
FR-13	The system could support autonomous navigation to a specified area.	Partial autonomy is helpful but not essential as operator can drive manually. Also depends on FR-11 to be possible.	UC-13
FR-14	The system could support autonomous return to starting position.	Convenience feature; operator can drive back manually. Also depends on FR-13.	UC-14
FR-15	The system could allow the operator to manually log detected survivors.	Useful for mission documentation and coordination. However missions can work without logging, and this is a secondary use for the system.	UC-10
FR-16	The system could automatically capture location and timestamp when logging survivors.	Enhances logging functionality. If logging exists, metadata such as this should be included automatically rather than have the operator do it manually.	UC-10
FR-17	The system could allow users to view survivor detection logs.	Dependent on FR-15 and FR-16, and is necessary for them to have any value what so ever.	UC-10

Table 6: Could have functional requirements

Non-Functional

Must Have

ID	Requirement	Justification	Related
NFR-01	The system must operate on a portable, battery-powered computing platform.	As the rover must be mobile and deployable, the system must be lightweight enough to run on the limited resources available in such such scenario.	-

Table 7: Must have non-functional requirements

Should Have

ID	Requirement	Justification	Related
NFR-02	The control interface should be accessible using a standard web browsers without any software installation.	Emergency scenarios require rapid deployment, reduces time to setup rover. However a solution requiring installing software could still work.	UC-05
NFR-03	The video feed should maintain a minimum frame rate of 20 fps outside of human detection.	Good control performance can be achived with 20+ fps [23]. Usable but degraded experience below this.	FR-02

Table 8: Should have non-functional requirements

Could Have

ID	Requirement	Justification	Related
NFR-05	The human detection could achieve a balanced precision and recall (F1 $\geq$ 0.85).	Higher accuracy is better, but the operator provides active judgement. Lower accuracy is acceptable with human oversight.	FR-04

Table 9: Could have non-functional requirements

6 Implementation

6.1 Software Implementation

For this section we will go over the different software implementations of the system, like commuinication protocols, computer vision system, a* algorithm, and more.

System Architecture Overview

The rover software is structured as a three-layer architecture that separates presentation, application logic and hardware control. This separation was done due to the differing requirements of the components, as well as the hardware on which they are hosted on. The user interface requires visualization and interactivity, for which modern web technologies such as React are well suited. Pathfinding algorithms and computer vision require substantial processing power which can be provided by a Raspberry Pi. Meanwhile direct control of motors and interaction with sensors requires low-level hardware access, and is best performed using an embedded system such as the jetbot. The resulting system consists of a React web frontend, a Python FastAPI backend, and jetson firmware, with communication between them achieved through WebSockets and i2c

Layer 01: Web Frontend (React Application)

The presentation layer is a React-based Single Page Application accessed by the operator in a web browser. The operator can navigate to the IP address of the Raspberry Pi, from which the Pi serves the web application. Once all elements of the application have been loaded, the application runs client-side locally on the operators browser, rather than on the Raspberry Pi. The frontend provides a user interface from which the operator can control the rover and receive visualization and data from it.

A browser based solution using React was chosen for several reasons. The web-based approach eliminates the need for operators to download a custom desktop application, enabling access to the interface easily and instantly with any web browser regardless of device. To maintain a connection with the rest of the rover, a Single Page Application was required rather than a traditional multi-page application as a persistent connection would be interrupted by page reloads. React was selected as the UI library due to being designed specifically for this purpose. Its component-based architecture makes it possible to create reusable and self-container components that manage their own state and behavior, which is ideal for interactive interfaces such as the rover interface. It also provides tools for managing application state and updating the user interface with data, allowing for instant updates in response to user interaction or incoming data.

Layer 02: Backend Application (FastAPI Server)

The FastAPI application serves as the application logic layer. It runs as a Python server process on the Raspberry Pi, and is independent of any operator connected through the frontend. The backend handles most of the heavy internal logic and also fulfills the role of an intermediary between the user interface and the Arduino that directly controls the rover. Other tasks of the backend include state management, pathfinding using an A* algorithm, human detection with computer vision and position tracking using dead reckoning.

The choice of FastAPI for this purpose was driven by multiple considerations. The most important for this project is its built-in WebSockets support, which was judged to be the optimal protocol for communication between the frontend and backend. The WebSockets protocol is designed specifically for maintaining bi-directional, low-latency communication over a single, persistent connection between a client and a server, which is precisely what is needed for controlling a robot [24]. Whilst HTTP is used due to technical limitation, for the movement controll scheme, it presents several downsides in terms of latency and throughput when it comes to something such as video streaming, and websocket was therefore chosen for the remainding parts of the project. Among the many libraries and frameworks with WebSockets support, FastAPI stood out due to its simplicity and its great performance, which is paramount when working with the limited resources offered by an Raspberry Pi.

To complement FastAPI, the asyncio library was used to enable asynchronous operations. The importance of this choice will be discussed in later sections.

Layer 03: Jetbot (Physical Rover)

The Jetbot from nvidia serves as our rover for this project. It runs a python based websocket for the webcamera and a python based movement firmware in jupyter functioning over http requests, as jupyter doesn't support websockets. The choices made in the firmware, as well as how to run the firmware, unfortunately vastly came down to what was technically feasible, rather than what would've been ideal, thanks to several challenges, including but not limited to; I2C problems, motor problems, camera problems, storage allocation problems, etc. All where unfortunately issues that we did not cause, with the majority being direct hardware issues and thereby not being solveable by us

The jetbot was programmed to move on a grid, as to always keep it well alligned with our a* algorithm. Whilst this limited movement in manual mode to 90 degree turns, and movement in 20cm increments, it greatly simplified the code needed to integrate autonomous driving into the bot

Communication

Having covered the overall architecture of the system, this section will cover the communication protocols that enable the different components of the rover to send data between each other. The communication between different processing units is essential to allow the separate parts to function as one cohesive system. For a reconnaissance rover the communication is especially important. As the robot needs to be controlled and considering the nature of the environments this rover is meant to be deployed it, having reliable and quick communication is essential to ensure minimal latency.

The majority of the system, including frontend, backend, cv system and camera feeds communicate over websockets. this is to have one fast and reliable unified channel for almost* all of our communicative needs.

Originally we wanted to use websocket for the last mile communication between backend and movement firmware as well, however as jupyter does not support websocket interupts, and jetson-io unfortunately can't run on the jetson's native python installation, we had to compromise and use http requests instead for this, as the native python installation cannot be upgraded due to the debian distribution being more than 8 years outdated. This has lead to some undesired fragmentation, but on the contrary has lead to movement commands being provided over a stable TCP connection, rather than the much less error resitant udp that websockets use

WebSockets Jetbot Implementation

Websockets are, as mentioned previously, used to relay the camera stream from the jetbot over to our raspberry pi. It does so with 30 frames per second target (real numbers will vary due to network jitter, and performance overhead). This is accomplished by a very simple 1 way websocket hosted on the jetbot, that simply listens for a connection, and starts streaming the camera feed once a connection has been made. As it's UDP there's no guarantee that all data will make it, but this is of lesser importance since even a few missed frames won't have any significant impact on either operators ability to maneuver the rover, nor the computer vision system as it's cleverly designed to simply take the latest recieved image

WebSockets Frontend Implementation

Additionally the frontend and backend communicates via a websocket running on localhost on the raspberry pi. On the frontend, the WebSocket implementation is done as a custom React hook `useRoverWebSocket()`, which encapsulates all connection management and message routing. By having everything centralized in this hook instead of it being scattered throughout the codebase, UI components remain focused on user interaction while the hook handles communication and state management behind the scenes. It also allows for one single WebSocket connection for handling all communication, whereas handling it in the individual components would most likely lead to creating individual connections for each. In Listing 1, part of the initial setup for the hook can be seen.

```ts
export function useRoverWebSocket() {
    const [pathData, setPathData] = useState<PathData>(null); ❶
    const [cameraData, setCameraData] = useState<CameraData | null>(null);
    const [isConnected, setIsConnected] = useState(false);
    const [roverStatus, setRoverStatus] = useState<RoverStatusData>({speed: "100%",
    human_detection: false, autonomous_driving: false});
    const ws = useRef<WebSocket | null>(null);

    useEffect(() => { ❷
        // setup websocket connection
        ws.current = new WebSocket("ws://localhost:8000/ws");
…                                   { handler setup }
        return () => {
            ws.current?.close();
        }
    }, []); ❸
```

Listing 1: WebSockets hook setup (`useRoverWebsockets.ts`)

All data which may be received from the backend is stored in state variables inside the hook ❶. This way components that need this data can simply subscribe to any of the state variables, and automatically update whenever new data arrives and the state gets updated. After these state variables, the WebSockets connection is setup within a `useEffects()` hook ❷. This hook, when used with an empty dependency array `[]` ❸, runs one single time on mount. Practically this means that the hook will attempt to initialize a connection with the backend as soon as the application loads.

Inside the `useEffect()` hook, it is also defined what should be done when a message is recieved over the WebSockets connection. As previously mentioned when covered the backend implementation of WebSockets communication, two different data formats can be received here: Binary camera frames for the video feed, or JSON for anything else. How this is handled can be seen in Listing 2.

```ts
…                                   { inside useEffect() }
        ws.current.onmessage = (event) => { ❶
            if (event.data instanceof Blob) { ❷
                const url = URL.createObjectURL(event.data);
                setCameraData({ image: url }); ❸
                return;
            }
        }
```

Listing 2: WebSockets hook setup (`useRoverWebsockets.ts`)

The `onmessage` method on the WebSocket object determines the behavior upon receiving a message. By changing this, we can change the behavior ❶. When receiving a message, its determine if the data has the format of a blob (binary large object)❷, in which case its identified as a camera frame, and the corresponding state variable is set to the new frame ❸. A flowchart showcasing this routing behavior can be seen below on Figure 6

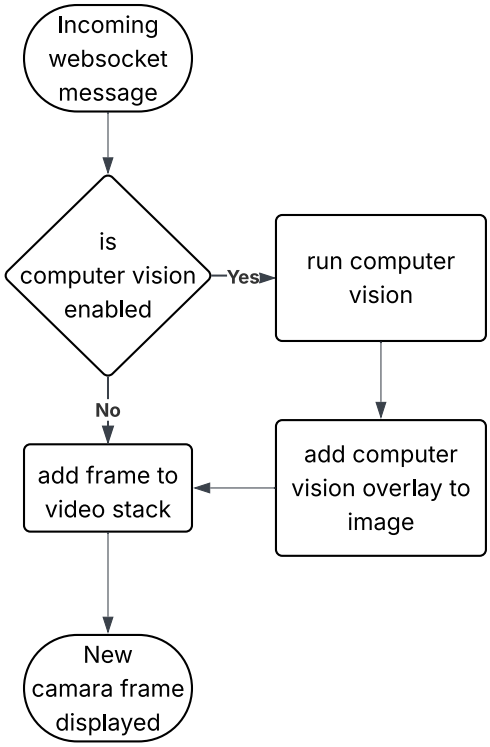


Figure 6: Frontend message routing flowchart

Websockets Backend Implementation

this section will explain the backend websocket

```py
@app.websocket("/ws")
async def main(ws: WebSocket):
…                                { checks and initalization of values }
 queues.send_queue.put({
        "type": "rover_status_data",
        "data": {
            "speed": getattr(RoverState, "speed", 0),
            "human_detection": visuals_handler.human_detection_enabled,
        },
    })

    receive_task = asyncio.create_task(receive(ws))
    send_task = asyncio.create_task(send(ws))

    try:
        done, pending = await asyncio.wait(
            [receive_task, send_task], return_when=asyncio.FIRST_COMPLETED
        )
        for task in pending:
            task.cancel()
        await asyncio.gather(*pending, return_exceptions=True)
        await asyncio.gather(*done, return_exceptions=True)

    except Exception as e:
        print(f"[ws] Error: {e}")
    finally:
        print("[ws] Cleaning up")
```

Listing 3: sartup of backend websocket code(`main.py`)

this code starts the websocket and listens to it to see when it recieves something and thereby when to start broadcasting the camerafeed

```py
async def receive(ws: WebSocket):
    try:
        while True:
            data = await ws.receive_json()
            print("[ws] Received Data:", data)

            msg_type = data.get("type")

            if msg_type == "move_rover":
                handle_move(data.get("data"))

            elif msg_type == "change_speed":
                if "handle_speed" in globals():
                    handle_speed(data.get("data"))

            elif msg_type == "enable_human_detection":
                payload = data.get("data", {})
                if payload.get("enable", False):
                    visuals_handler.enable_human_detection()
                else:
                    visuals_handler.disable_human_detection()

            elif msg_type == "sensor":
                handle_sensor(data)

            elif msg_type == "enable_autonomous_driving":
                payload = data.get("data", {})
                enable = payload.get("enable", False)

                if enable:
                    print("[ws] Autonomous driving ENABLED")
                    RoverState.recalculate_path()
                    RoverState.start_autonomous()
                else:
                    print("[ws] Autonomous driving DISABLED")
                    RoverState.stop_autonomous()
            else:
                print(f"[ws] Unknown message type: {msg_type}")

    except WebSocketDisconnect:
        print("[ws] Receive connection lost")
        raise
    except asyncio.CancelledError:
        print("[ws] Receive task cancelled")
        raise
    except Exception as e:
        print(f"[ws] Receive error: {e}")
        raise
```

Listing 4: recieving BE websocket (`main.py`)

when the websocket recievs data it checks to see if it fits into a existing catagory and executes the apropriate code.

```py
async def send(ws: WebSocket):
    try:
        while True:
            # ─── Camera frames ───
            jpeg_bytes = None
            while not queues.camera_queue.empty():
                jpeg_bytes = queues.camera_queue.get_nowait()

            if jpeg_bytes is not None:
                if visuals_handler.human_detection_enabled:
                    jpeg_bytes = await visuals_handler.run_human_detection_on_jpeg(jpeg_bytes)
                await ws.send_bytes(jpeg_bytes)

            # ─── Other queued data ───
            while not queues.send_queue.empty():
                data = queues.send_queue.get_nowait()
                try:
                    await ws.send_json(data)
                    print(f"[ws] sent JSON type={data.get('type')}")
                except Exception as e:
                    print(f"[ws] failed sending JSON ({data.get('type')}): {e}")

            await asyncio.sleep(0.01)

    except asyncio.CancelledError:
        print("[ws] Send task cancelled")
        raise
    except WebSocketDisconnect:
        print("[ws] Send connection lost")
        raise
    except Exception as e:
        print(f"[ws] Send error: {e}")
        raise
```

Listing 5: sending BE websocket (`main.py`)

this code looks at the data it has and sends it to the frontend.

HTTP

For movement an error resistent http tcp connection is used over port 5000 to let the jetbot know either the operators or autonomous driving algorithms intention. The jetbot does not know if it's a human or the computer telling it to move, and because of this the jetbot will also move in the same patterns. this means 90 degree turns ONLY, and 20cm movement intervals ONLY. This limitation guarantees we stay on a predetermined grid, and makes the implementation of the autonomous drivnig and A* systems much simpler, especially given the tecnical limitations in our positioning estimation

In order to move the rover, our backend on the Section 6.2 will send a `POST /move/<action>` request, where <action> is whatever operation we wish the rover to execute. as seen in Listing 20 the jetbot is programmed to, with the aid of timers and some hand tuned values, move precisely for x amount of time in order to fascilitate movement / turning. This is done as the jetbot does not have any native feedback from the motors. This means we can't reliably tell how far the jetbot have moved, and that floating point errors are more than likely to make the motor speeds drift eventually, leading to desync with A* expected position

I2C (Inter-Integrated Circuit)

I²C (Inter-Integrated Circuit) is a sleek, synchronous serial bus that lets multiple masters and slaves talk over just two wires—SDA for data and SCL for the clock. Its single-ended, bidirectional design makes wiring a breeze while still supporting robust multi-device communication, perfect for everything from sensor arrays to peripheral controllers.

- SDA: Data
- SCL: Clock

I2C is used in comunication between the Jetson and the Jetbot. However as this was pre-implemented by nvidia, we will not be touching on so, but rather be refering to nvidia's official documentation[16]

Rover State Management

In a robotics system such as this, maintaining a consistent state is important to ensure coordinated behavior. In the case of state management, having one single centralized system for managing state is advantageous as opposed to each part having their own state to manage. This however can be difficult when dealing with distributed systems, where constant communication to read the state is unideal. A solution to this is to have each component maintain its own local state, with one central state serving as the "source of truth", from where other local states are synchronized when necessary. In this project, the central state is managed on the backend inside the `RoverState` class. State synchronization happens outwards from the backend. When the state is updated, an observer pattern (detailed later) is used to automatically transmit the new state to the frontend via WebSockets.

For state management, the backend utilizes a singleton design pattern. This means that the class is ensured to have only one instance, with a global access point [25]. This pattern was chosen as the backend needs exactly one authoritative state, in which case the singleton pattern fits perfectly. And rather than passing a state object through every function call or similar, the single `rover_state` object can simply be refereed to from anywhere, greatly simplifying development. Below in Listing 6, the initialization of the state class can be seen.

```python
class RoverState:
    def __init__(self):
        # ———————————————————— configure these ————————————————————
        self.obj_grid = [
            [0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0],
            [0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
…                                       { more rows }
        ]

        self.current_position = [0, 0]
        self.destination = [7, 14]
        self.obj_theshold = 0
        self.grid_cell_size = 0.6

        # ———————————————————— dont touch these ————————————————————
        self.current_path = None
        self.speed = "100%"
        self.on_path_update = None
        self.tracker = PositionTracker(0, 0, "right")
        self.last_sensor_time = None
        self.on_drive_cmd = None
        self.on_path_update = None
        self.running = False
        self.dt = 0.2
        self.direction_order = ["up", "right", "down", "left"]
…                                           { … }
rover_state = RoverState() ❶
```

Listing 6: Rover state initialization (`state.py`)

The state variables are categorized by those that can be configured and those who can't. Among the configurable variables is the map grid, current position and destination on the grid and the size of each cell (translated to meters). In a more advanced iteration of the rover, most of these would be determined from outside the state, such as sensors or user input, however as a proof of concept they are simply set here. Among non-configurable variables are the current path (determined by the pathfinding algorithm) and current speed that are modified during runtime. Moreover `on_path_update` holds the function to call whenever the path changes (covered later), and `position_tracker` encapsulates dead reckoning calculations (detailed in). At the end the global singleton instance is created ❶.

When updating these state variables, it is best practice to not directly update them, but instead create dedication methods and call those instead. This is especially the case for a value update should also trigger one of more other actions. In Listing 7 two of these methods can be seen, `update_position` and `set_speed`. There are more than this, but they mostly follow the same pattern.

```python
…                                   { in RoverState }
def start_autonomous(self):
  if not self.running:
      Self.running = True
      asyncio.create_task(self._drive_loop())

def stop_autonomous(self):
    self.running = False

def update_position(self, position):
    rev_dir = {
        (-1, 0): "up",
        (1, 0): "down",
        (0, -1): "left",
        (0, 1): "right",
    }

    row, col = position

    dx = col - self.tracker.x
    dy = row - self.tracker.y


    self.tracker.y = row
    self.tracker.x = col

    print(rev_dir[(dy, dx)])
    self.tracker.change_direction(rev_dir[(dy, dx)])

    self.current_position = [row, col]

    self.recalculate_path()

```

Listing 7: Some of the setter functions (`state.py`)

The first function `start_autonomous()` starts the autonomous driving and the second function `stop_autonomous()` stops the autonomous driving.

The third function `update_position()` is used to, update the rovers position on the grid. This is pretty simply done by just setting the state variable to the one passed as argument. However it also calls the `recauclate_path()` method. As the rover is at a new position, the shortest path to its destination naturally changes too, and a new path has to be calculated.

The `recaculate_path` method is an abstraction of the underlying A* algorithm which is detailed in ***A****. The method adds on top of the algorithm by also updating state variables, and synchronizing with the frontend state. For the state synchronization an observer pattern is used, enabling the rover state to notify any interested components without being directly dependent on the WebSockets implementation [26].

```
…                                        { in RoverState }
def recalculate_path(self):                                                    py
…                                     { A* and state update }
    if self.on_path_update:
        path_data = {❶
            "grid": self.obj_grid,
            "path": [[r, c] for (r, c) in self.current_path],
            "start": self.current_position,
            "destination": self.destination,
        }
        asyncio.create_task(self.on_path_update(path_data))
```

Listing 8: State synchronization (`state.py`)

To achive this, the `on_path_update` callback function is called with the new updated state variables (in this case new path data) as argument ❶. The callback function to call is registered during the backend initialization, as seen in Listing 9

```
…                                              { … }
    # set rover up to send path update to ws send queue when the path updates   py
    async def send_path_update(data): ❶
        await send_queue.put(data)

    rover_state.on_path_update = send_path_update

    # initial path calculation
    rover_state.recalculate_path() ❷

    # send current rover state
    await send_queue.put({ ❸
        "type": "rover_status_data",
        "data": {
            "speed": rover_state.speed,
            "human_detection": visuals_handler.human_detection_enabled
        }
    })
```

Listing 9: Observer registration (`main.py`)

In this case the callback function is very simple and just puts the data into the send queue to be send over websockets ❶. However it is easily extensible and further instructions can be added to the callback if more needs arrise. After the callback function is set up, the backend does calculation of the initial path ❷, and sends the rest of the current state as well ❸.

A^*

As mentiond in A^* it has a focused search that looks at the values each node and moves the the node with the lowest h value if posible.

```py
class Cell:
    def __init__(self):
        self.parent_i = 0  # Parent cell's row index
        self.parent_j = 0  # Parent cell's column index
        self.f = float('inf')  # Total cost of the cell (g + h)
        self.g = float('inf')  # Cost from start to this cell
        self.h = 0  # Heuristic cost from this cell to destination
```

Listing 10: node prameters (`a_star.py`)

Here we create a class that contains the values that each node contains.

```py
def is_valid(row, col):
    return (row ≥ 0) and (row < ROW) and (col ≥ 0) and (col < COL)


def is_unblocked(objGrid, row, col, objThreshhold):
    return objGrid[row][col] ≤ objThreshhold


def is_destination(row, col, dest):
    return row == dest[0] and col == dest[1]


def calculate_h_value(row, col, dest):
    return ((row - dest[0]) ** 2 + (col - dest[1]) ** 2) ** 0.5
```

Listing 11: checks and helper funktions (`a_star.py`)

these are the validations that are used in the actual a*.

```python
def trace_path(cell_details, dest):
    print("The Path is ")
    path = []
    row = dest[0]
    col = dest[1]

    # Trace the path from destination to source using parent cells
    while not (cell_details[row][col].parent_i == row and cell_details[row][col].parent_j == col):
        path.append((row, col))
        temp_row = cell_details[row][col].parent_i
        temp_col = cell_details[row][col].parent_j
        row = temp_row
        col = temp_col

    # Add the source cell to the path
    path.append((row, col))
    # Reverse the path to get the path from source to destination
    path.reverse()

    # Print the path
    for i in path:
        print("->", i, end=" ")
    print()
    return path
```

Listing 12: traces the path that A* finds (`a_star.py`)

this is the final fuction that is called in the a* function it goes through the array of searched nodes in reverse order and returns an array of the path

```py
def a_star_search(objGrid, src, dest, objThreshhold):
…                        { preliminary checks and Initialization }
    # Main loop of A* search algorithm
    while len(open_list) > 0:
        # Pop the cell with the smallest f value from the open list
        p = heapq.heappop(open_list)

        # Mark the cell as visited
        i = p[1]
        j = p[2]
        closed_list[i][j] = True

        # For each direction, check the successors
        directions = [(0, 1), (0, -1), (1, 0), (-1, 0)] #(1, 1), (1, -1), (-1, 1), (-1, -1) ← diagonale retninger
        for dir in directions:
            new_i = i + dir[0]
            new_j = j + dir[1]

            if is_valid(new_i, new_j) and is_unblocked(objGrid, new_i, new_j, objThreshhold) and not closed_list[new_i][new_j]:
                # If the successor is the destination
                if is_destination(new_i, new_j, dest):
                    # Set the parent of the destination cell
                    cell_details[new_i][new_j].parent_i = i
                    cell_details[new_i][new_j].parent_j = j
                    print("The destination cell is found")
                    # Trace and print the path from source to destination
                    path = trace_path(cell_details, dest)
                    found_dest = True
                    return path
                else:
                    # Calculate the new f, g, and h values
                    obj_new = 1
                    g_new = cell_details[i][j].g + 1.0
                    h_new = calculate_h_value(new_i, new_j, dest)
                    f_new = (g_new + h_new)**obj_new

                    if cell_details[new_i][new_j].f == float('inf') or cell_details[new_i][new_j].f > f_new:
                        heapq.heappush(open_list, (f_new, new_i, new_j))
                        cell_details[new_i][new_j].obj = obj_new
                        cell_details[new_i][new_j].f = f_new
                        cell_details[new_i][new_j].g = g_new
                        cell_details[new_i][new_j].h = h_new
                        cell_details[new_i][new_j].parent_i = i
                        cell_details[new_i][new_j].parent_j = j
```

Listing 13: the a* algorithm code(`a_star.py`)

this is the function that does the actuall pathfinding. As mentiond in the A^* section it itterates through a grid and looks at all adjecent nodes to the curent node

big-O

Since heapq is used and both heapq.heappush and heapq.heappop have a Big-O of log(n), where n is the number of nodes in the grid, and because the algorithm loops through the grid, the time complexity of the A* algorithm is O(n log n)

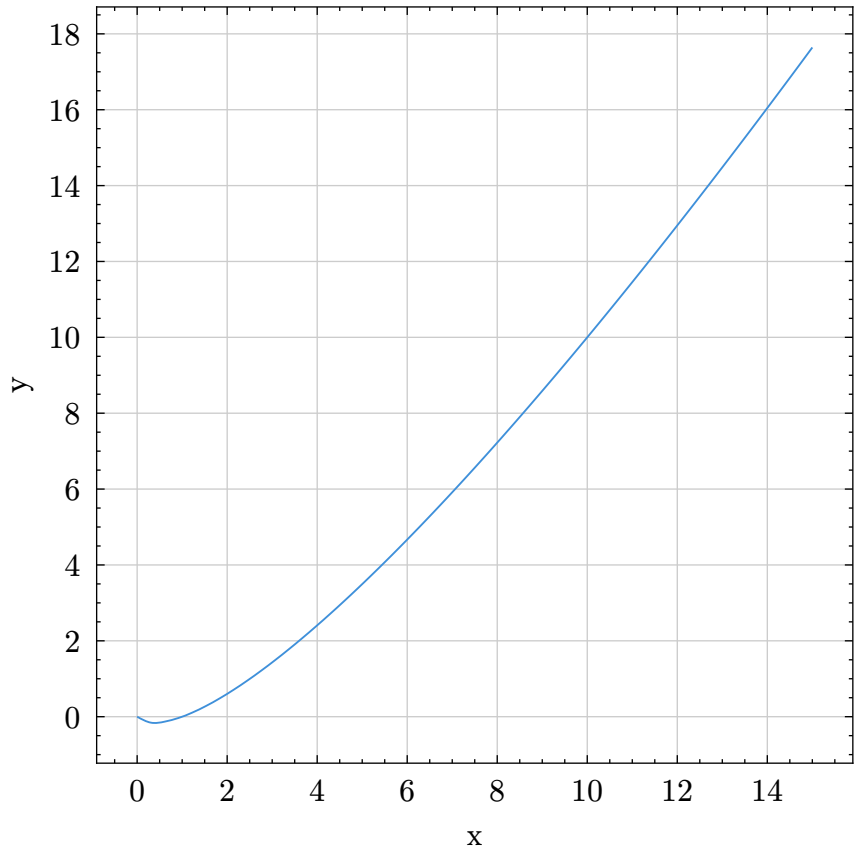


Figure 7: Plot of $F(x) = x\log(x)$

This means the time complexity is larger then a linear O(n) after n reaches 10 but faster until n reaches 10 Figure 7

position tracking

for the autonomous driving we needed a way to track the possition and direction of the rover so we made this simple script

```py
class PositionTracker:
    # grid directions: (dr, dc)
    directions = {
        "up":    (-1, 0),
        "down":  (1, 0),
        "left":  (0, -1),
        "right": (0, 1),
    }

    def __init__(self, row, col, direction):
        if direction not in self.directions:
            raise ValueError(f"Invalid direction: {direction}")

        self.y = row
        self.x = col
        self.direction = direction

    def move(self):
        dr, dc = self.directions[self.direction]
        self.y += dr
        self.x += dc

    def move_back(self):
        dr, dc = self.directions[self.direction]
        self.y -= dr
        self.x -= dc

    def change_direction(self, direction):
        if direction not in self.directions:
            raise ValueError(f"Invalid direction: {direction}")
        self.direction = direction

    def position(self):
        return [self.y, self.x]

    def __str__(self):
        return f"pos = ({self.y},{self.x}), direction = {self.direction}"
```

Listing 14: position tracker code(`position_tracker.py`)

this it the script for tracking the possition and direction of the rover

autonomous driving

this is the code section for the autonomus driving

```
…                                { initalize important values }
def send_movement_command_blocking(action: str):                    py

    if action not in MOVE_ACTIONS:
        print(f"[Movement] Invalid action: {action}")
        return

    url = f"{MOVEMENT_URL}/{action}"

    try:
        response = requests.post(url, timeout=1)
        response.raise_for_status()
        print(f"[Movement] Sent action '{action}', response: {response.text}")
    except Exception as e:
        print(f"[Movement] Failed to send action '{action}': {e}")


async def send_drive_cmd(action: str):
    await asyncio.get_running_loop().run_in_executor(None,
    send_movement_command_blocking, action)
```

Listing 15: autonumus driving helper fuctions (`state.py`)

these helper functions `send_movement_command_blocking()` and `send_drive_cmd()` work together since `send_drive_cmd()` is supposed to execute the `send_movement_command_blocking()` function that sends the movement command to the jetbot.

```py
async def _drive_loop(self):
        while self.running:
            if not self.current_path:
                await asyncio.sleep(self.dt)
                continue
…                           { initalize important values }

            current_index = self.direction_order.index(self.tracker.direction)
            desired_index = self.direction_order.index(desired)
            delta = (desired_index - current_index) % 4

            if delta == 0:
                move_action = "forward"
            elif delta == 2:
                move_action = "backwards"
            elif delta == 1:
                # Turn right once, then forward
                if self.on_drive_cmd:
                    await self.on_drive_cmd("right")
                self.tracker.change_direction(desired)
                move_action = "forward"
            else:  # delta == 3
                if self.on_drive_cmd:
                    await self.on_drive_cmd("left")
                self.tracker.change_direction(desired)
                move_action = "forward"

            # Update tracker position locally
            if move_action == "forward":
                self.tracker.move()
            else:
                self.tracker.move_back()

            if self.on_drive_cmd:
                await self.on_drive_cmd(move_action)

            # Update state position
            self.current_position = [self.tracker.y, self.tracker.x]

            self.recalculate_path()
            await asyncio.sleep(self.dt)

```

Listing 16: autonumus driving fuctions (`state.py`)

This code runs the autonomous driving system based on the path outputted from A*. As precise movement is not possible Section 8.3 the code runs on an assumption of precise and consistent movement from the jetbot. It is programmed to move a fixed distance for every command, and rotate 90 degrees for every command, leading to it, in theory, always staying on grid. autonomous driving will always continue moving forward untill it encounters a turn, at which point it'll cancel all movement, rotate 90 degrees, and continue movement as planned.

Human Detection System

As a reconnaissance rover, one of its main objectives is locating humans in disaster environments. It is mainly the operators' task to observe the video feed and determine whether if any humans can be found. To aid them in this task, it was decided to equip the rover with a computer vision model trained to detect humans, vastly improving the operators' ability to find them. For this purpose, a YOLO-based human detection system was implemented, specifically optimized for the Raspberry Pi's limited computing capabilities. Implementation-wise, the human detection system was also be integrated with the ordinary camera feed system, and it should be possible to seamlessly enable and disable the human detection.

Model Selection and Training

Considering the constraints of the hardware, selecting the appropriate model is especially important. Balancing accuracy with performance is difficult, given that low performance would make control of the rover challenging due to a low frame rate, while low accuracy would make the model less capable of detecting humans. For this project, the YOLO (You Only Look Once) family of models was chosen. The YOLO model is an object-detection system based on convolutional neural networks [27]. It was chosen because of its single-stage detection architecture, which prioritizes inference speed while maintaining competitive accuracy. It is known for its speed, which makes it more suitable for embedded systems than more computationally intensive models such as CNNs. Furthermore, the Ultralytics framework, which was used to work with the model, greatly simplifies the training process and makes it easy to integrate the model into another project [28]. Besides the model family, it was also necessary to choose the specific model. YOLO models come in various sizes, with smaller sizes offering faster inference time at the cost of lower accuracy. The smallest model, "nano," was chosen for this project, as any larger model would most likely be too computationally expensive. Additionally, the YOLO11n model was chosen in favor of the newer YOLO12n model due to superior inference performance [29]. YOLO12 is also primarily designed for benchmarking and research, and Ultralytics themselves, the developer of both model variations, recommend using YOLO11 instead if stable training, predictable memory usage, and optimized CPU inference are needed [30].

Besides selecting a suitable model variant, selecting the correct training data to train the model is equally important for ensuring its accuracy in detecting humans. During research, suitable datasets were narrowed down to two candidates: "AiTV Disaster Scenarios" [31] and "C2A Disaster Scenarios" [32]. Both datasets are specialized in humans located in disaster environments, making them appropriate for the purpose of the rover. In the end AiTV was chosen due to superior accuracy.

The YOLO11n model was trained on the AiTV dataset for 100 epochs, and achieved strong performance across object detection metrics. The following figures showcase the achieved performance. When reading these graphs, it is useful to know what the different metrics actually mean. Three different metrics are measured across the graphs [33]:

- Precision: Measures how many of the predicted positive instances are actually correct.
- Recall: Measures how many of the actual positive instances were correctly identified by the model.

- F1: This is the harmonic mean of precision and recall. It combines precision and recall into a single metric, making it useful to get an overall idea of the performance of a model. It can also be used to find the optimal confidence point to operate after.

The precision-confidence curve Figure 8 shows that the model achieve close to perfect precision at high confidence tressholds (>0.9), indicating that detections flagged with high confidence are highly reliable Figure 8. This information is useful for filtering false positives, as anything with a confidence over 0.9 almost certainly can be trusted.

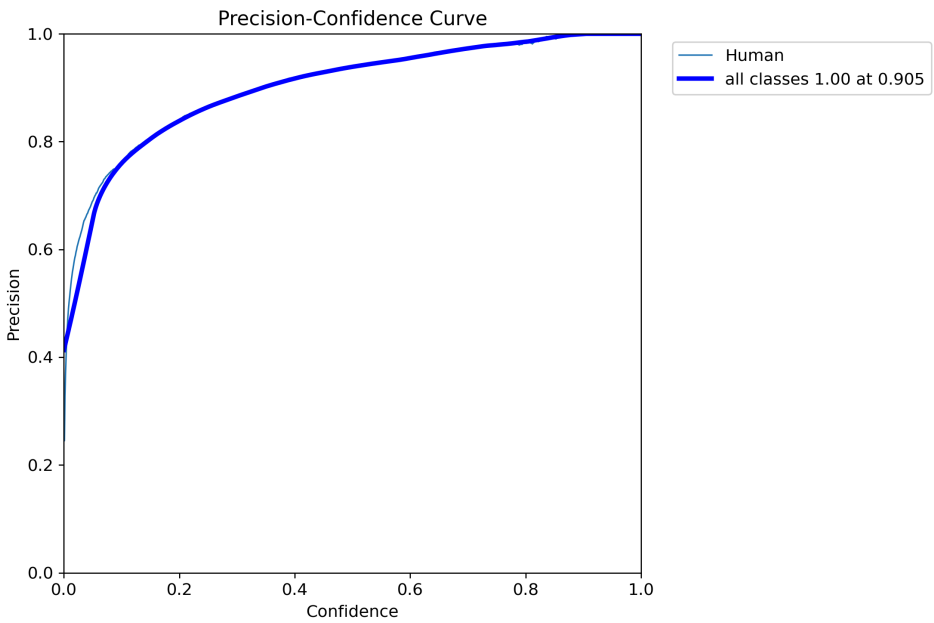


Figure 8: precision is predicted positive instances are actually correct

The recall-confidence curve Figure 9 demonstrates that recall reaches 0.95 at low confidence thresholds but degrades as the threshold increases. This behavior means that at higher confidence thresholds, more positive instances are missed. For rescue applications where missing a human is more critical than occasional false alarms, this means operating at lower confidence thresholds to maximize recall is preferable.

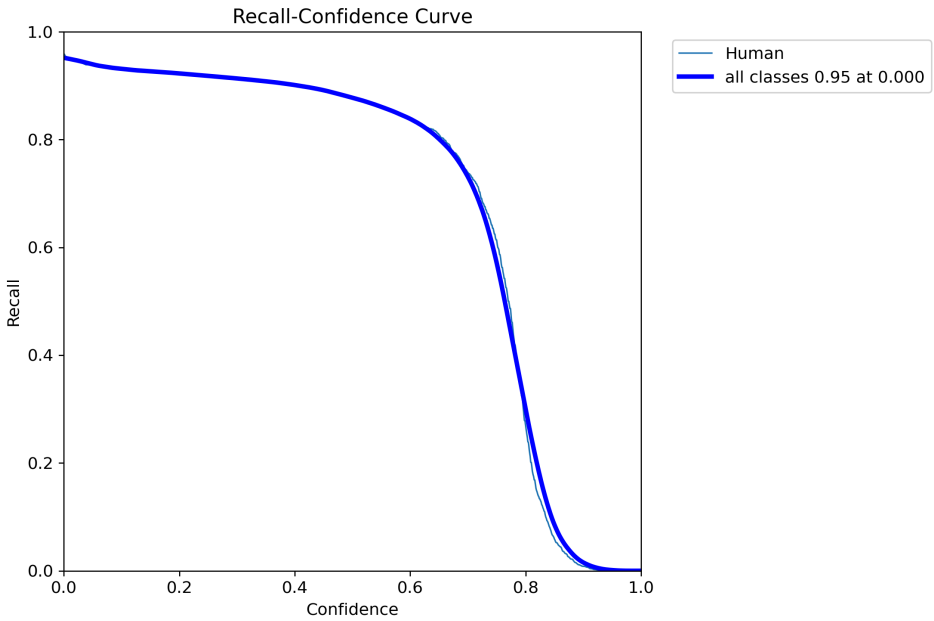


Figure 9: Recall is actual positive instances were correct

In the Precision-Recall curve Figure 10 it can be seen that the model achieved a mean Average Precision (mAP) of 0.930 at IoU threshold 0.5, indicating strong overall performance [33]. The large area under the curve means that the model maintains high precision across the majority of the recall range.

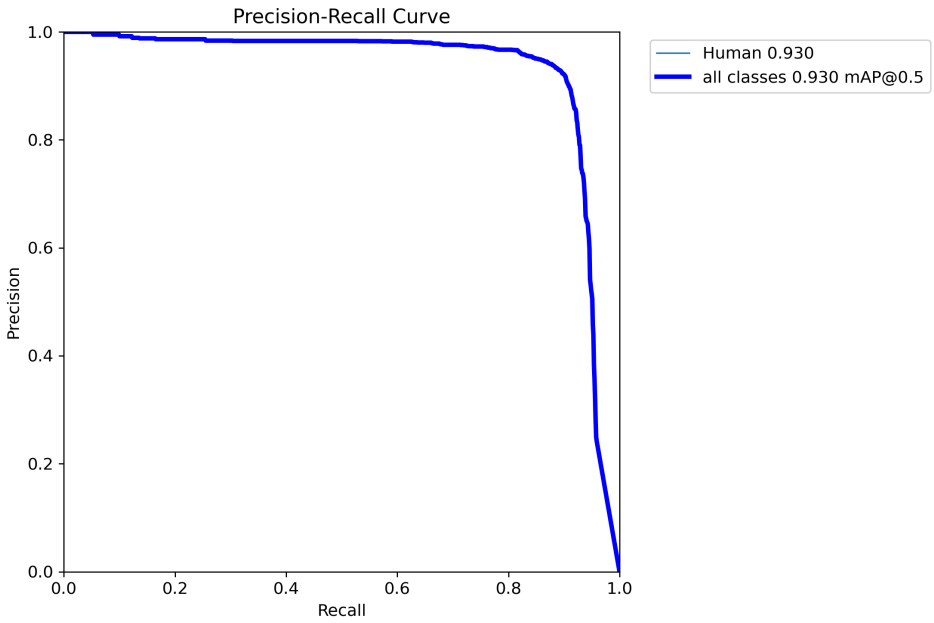


Figure 10: PR Curve plots precision against recall

The F1 confidence curve Figure 11 identifies the optimal operating point at confidence threshold 0.432, where the F1 score peaks at 0.91. This value can be used as a practical confidence threshold that balances false positives and false negatives.

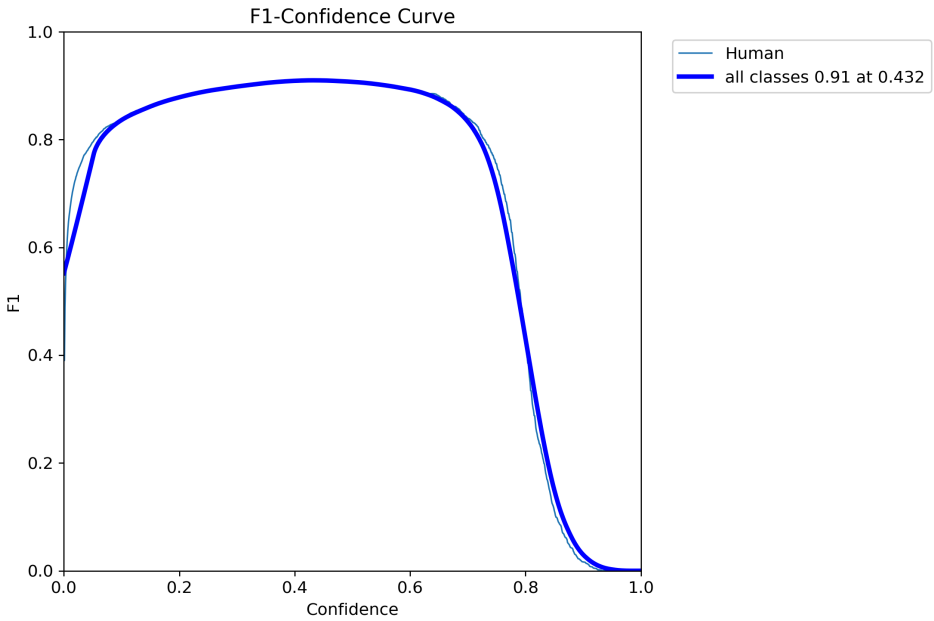


Figure 11: mean of precision and recall

Implementation

Implementing the neural network model on the Raspberry Pi requires effective integration with the existing backend. The way the model is formatted can also greatly influence its performance, so choosing the correct format for a Raspberry Pi is crucial for ensuring optimal inference speed. This subsection details the the technical implementation of the computer vision model on the backend, as well as optimizations made to ensure adequate inference performance.

YOLO models can be exported in many different formats which varying performance. Conveniently enough, Ultralytics already made a comparison between the different formats using the exact same model variation we use (YOLO11n) on the exact Raspberry Pi the backend is implemented on [34]. This makes choosing the best format rather trivial. The comparison can be seen on Figure 12

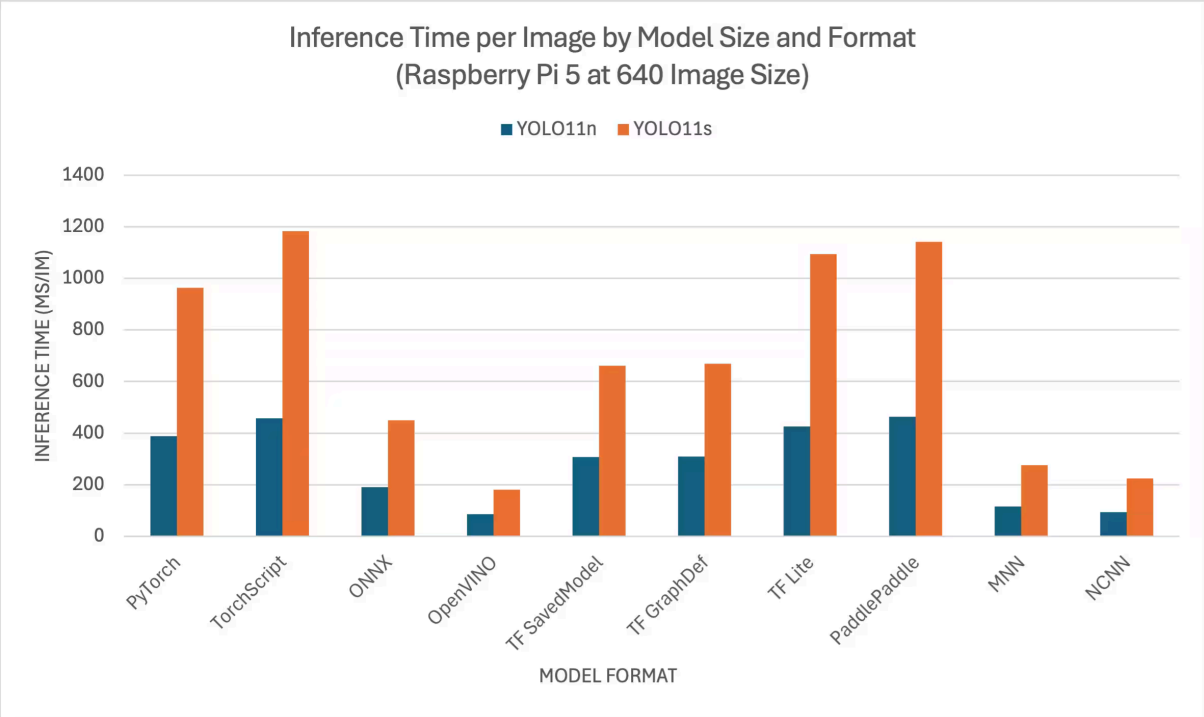


Figure 12: Benchmark of different model formats [30]

As can be seen in the comparison, there is a large difference in inference time across formats, highlighting the importance of chosing the correct one. Among the fastest formats are OpenVINO, MNN and NCNN. Among these, NCNN was chosen due to being recommended by Ultralytics, and having almost identical performance compared to the two other top contenders.

For the actual implementation on the backend, model inference was handled using the `VisualsHandler` class, which also handles camera management and frame encoding. Similarly to the state management class, `VisualsHandler` also follows the singleton design pattern with one single global access point. Below in Listing 17 the initialization of this class can be seen.

```py
class VisualsHandler:
    def __init__(self):
        self.cap = None
        self.camera_active = True
        self.model = None
        self.human_detection_enabled = False

    def initialize_camera(self):
        if self.cap is not None:
            self.cap.release()
            self.cap = None

        try:
…                                   { error handling }
            self.cap.set(cv2.CAP_PROP_FRAME_WIDTH, 960)
            self.cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 540)
            self.cap.set(cv2.CAP_PROP_FPS, 30)
            self.cap.set(cv2.CAP_PROP_BUFFERSIZE, 1)
            self.camera_active = True
            return True
```

Listing 17: Camera Initialization (`visuals_handler.py`)

The `initialize_camera` method is used to set the camera up using the `cv2` library. The resolution is set to 960×540, as higher resolutions leads to unacceptable performance when human detection is enabled. The target framerate is set to 30 fps, and the buffer size is set to 1 to reduce latency and only store the latest frame. The initialization also handles errors related to the camera being opened.

Initializing the human detection model, running the model on a frame and generating frames are each done in its own method. The most important parts of these methods can be seen in Listing 18.

```
…                                   { in VisualsHandler }
    def initialize_human_detection(self):                                   py
        try:
            self.model = YOLO("./visual/yolo11n_ncnn_model") ❶
            print("[ws] YOLO model loaded successfully")
            return True
…                                           { … }
    async def run_human_detection(self, frame):
        try:
            results = await asyncio.to_thread( ❷
                self.model,
                source=frame,
                imgsz=640,
                verbose=False,
            )

            return results[0].plot() ❸
…                                           { … }
    async def generate_frame(self):
        if not self.camera_active or self.cap is None:
            return None

        ret, frame = await asyncio.to_thread(self.cap.read) ❹
        if not ret:
            return None

        if self.human_detection_enabled:
            frame = await self.run_human_detection(frame) ❺

        _, buffer = cv2.imencode('.jpg', frame, [cv2.IMWRITE_JPEG_QUALITY, 60]) ❻
        return buffer.tobytes()
```

Listing 18: Camera Initialization (`visuals_handler.py`)

The `initialize_human_detection()` method sets up the YOLO model so it is ready to run inference. This is made easy with the Ultralytics framework, as you simply call `YOLO` with the path to the exported model ❶. With the model running, the `run_human_detection()` method can be called to run inference on a camera frame. This is also made easy by simply running the model with the frame as argument, alongside the image size ❷. On the result, the `plot()` method can be called in order to create an overlay showing any human detected, necessary for displaying the results visually ❸. This method is used in the `generate_frame()` method, which is the higher level function used to generate frames. First it captures a frame from the camera ❹, from which it runs this frame through the human detection model if human detection is enabled ❺. Afterwards it encodes the frame as a JPEG file, and returns the raw binary representation, ready to be sent to the frontend and displayed ❻.

The modular implementation of the vision system, with a separate singleton class encapsulating all visual systems, allow for easy extensions of any further functionality should the need arise.

6.2 RPI4B

For the computational part of the project, we decided to use a raspberry pi 4B. This wasn't ideal, as a raspberry pi 5 would've had more horse power to run the computer vision system at a higher FPS, unfortunately the unit we got was partially defective, and acquiring a new one proofed difficult. As compensation for the worse performance, the system was overclocked to 2.2GHz, and tuned to perform as good as possible, given the hardware. For deployment of our software we used docker containers. One for the whole backend (computer vision, websockets, http and A*), and one for the frontend. To make it easier in a development environment both ran with `--network=host` so they could talk to each other over `localhost`

6.3 jetbot

A jetbot is a small robotics kit developed by nvidia, to inspire and educate young people to get into robotics. It features an expansion board that interfaces with the motors, and batteries, and a jetson compute base, featuring a A57 cortex CPU, and a 128 cuda cores graphics unit. [16], [35] Our specific unit additionally had 4GB of RAM, as we didn't know the limitations of the system back then, and therefore thought we could run everything on it. We could not

In preparation for running the jetbot, we had to circumvent one mayor issue; Nvidia has set an articial limit on the sd card size on 22GB, that unfortunately was not adequate for the amount of packages we would need to install to get our camera websockets up and running. In order to circumvent this we had to edit the image, and delete the directory `/usr/bin/nvidia/nvidia-resizefs` as well as changing a few flags in the bootloader to get it to accept the frankenstein we had created

Then we could simply run `fdisk` on our sd card to expand the partition, and later `resize2fs` to properly make it a 58.9GB partition

camera websocket

In order for us to remotely control the rover, as well as run computer vision on the environment, we would need a connection to a camera onboard The original intention was to use the onboard jetbot camera, till we discovered that our sensor had been smashed prior to acusition.

We therefore decided to strap an ordinary webcamera to the front, and interface with it via the regular `/dev` in linux. As we did not have enough computational power on the jetbot to run all of our software, along with the jetbot's software suited being too outdated, and depricated, to even support much of the software, we decided the best cause of action would be using a UDP websocket to communicate back to a compute base;

```py
CAMERA_DEVICE = "/dev/video0"
WS_HOST = "0.0.0.0"
WS_PORT = 8765
JPEG_QUALITY = 80
FPS = 30


@asyncio.coroutine
def stream_camera(websocket):
    cap = cv2.VideoCapture(CAMERA_DEVICE)
    if not cap.isOpened():
        raise RuntimeError("Could not open webcam: {}".format(CAMERA_DEVICE))

    encode_params = [int(cv2.IMWRITE_JPEG_QUALITY), int(JPEG_QUALITY)]

    try:
        while True:
            ret, frame = cap.read()
            if not ret:
                break

            # Encode frame as JPEG
            success, jpg = cv2.imencode(".jpg", frame, encode_params)
            if not success:
                continue

            # Send as binary WebSocket message
            yield from websocket.send(jpg.tobytes())

            # Control FPS
            yield from asyncio.sleep(1.0 / float(FPS))

    except websockets.exceptions.ConnectionClosed:
        pass
    finally:
        cap.release()
```

Listing 19: Websocket exposer for jetbot camera feed (`CameraFeedExposer.py`)

the code depicted above shows how the websockets listens on all addresses on port 8765, and if a connection is established it broadcasts the camera's resolution (fhd) at 30 fps, encrypted in jpeg. Whilst it's a major security risk to have it listen to all addresses, this was deemed a negliceble risk in our controlled testing environment, as we ran bot and computational center Section 6.2 on an airgapped network

movement firmware

The movement on the jetbot can be very unprecise. It has way to violent acceleration in comparison to the wheels relatively low grip strength, and the motors has no feedback to the jetbot, leading to the rotational speed drifting slightly apart. We initially wanted to overcome this obstacle by implementing an IMU, that could give us precise feedback on how we moved. Unfortunately, the BMI160 did not seem to act as we wanted, and we therefore fell back to carefull manual tunning of the motors to get them as close to straight as possible. Whilst we understand that "close" can still become a huge margin of error over the span of kilometers, this was deemed "good enough" for a Proof of concept who would only ever drive 20meters.

```
…                                                    { … }
class Move:                                                                    py
    @staticmethod
    def stop():
        print("[MOVE] stop")
        robot.stop()

    @staticmethod
    def move_forward():
        print("[MOVE] forward")
        robot.set_motors(0.2, 0.19)
        sleep(0.72)
        robot.stop()

    @staticmethod
    def move_backward():
        print("[MOVE] backward")
        robot.set_motors(-0.2, -0.19)
        sleep(0.52)
        robot.stop()

    @staticmethod
    def move_right():
        print("[MOVE] left")
        robot.set_motors(0.105, -0.1)
        sleep(0.86)
        robot.stop()

    @staticmethod
    def move_left():
        print("[MOVE] right")
        robot.set_motors(-0.105, 0.1)
        sleep(0.86)
        robot.stop()
```

Listing 20: Jetbot movement firmware implemented in jupyter notebook

to accomodate the autonomous driving part, the firmware was written to drive a set distance upon input (20 cm). Now due to the lack of feedback, this had to be an estimation based upon the speed of the jetbot, which is less than ideal. we did calculate however that at the speeds specified it would take .72 seconds to move forward, and .86 seconds to turn 90 degrees

This movement would ensure we always stayed on grid, and that A* would have an as easy of a time as possible in navigating our rover. This also meant that any human intervention would have to stay on the same grid, and thereby could not accidentally throw A* off

7 Testing

7.1 Testing Approach

This chapter presents the tests performed to ensure that the rover meets its functional and non-functional requirements from Section 5.4. Testing is divided into two phases to ensure both component-level and integrated system functionality.

- **Preliminary Tests:** Individual subsystems were all tested in isolation, or integrated to other select subsystems to verify foundational functionality before full integration was tested. These tests validate core modules independently or in limited integration, making it possible to quickly identify defects and errors before full system tests
- **System Tests:** The complete prototype is tested end-to-end with complete workflows under realistic conditions. Each of these tests verify multiple requirements simultaneously, and focuses on usage scenarios where all necessary subsystems work together.

Testing Environment

Component	Description
Platform	NVIDIA JetBot
Network	-
Operator Device	Standard laptop running Firefox browser
Test areas	Various indoor environments

Table 10: Testing Enviorment

Test Documentation Format

Each test is documented with the following:

- **Objective:** What the test verifies
- **Related Requirements:** What requirements in Section 5.4 the test can be traced to
- **Test Environment (if applicable):** Specific setup details if they deviate from the environment specified earlier
- **Procedure:** Step-by-step method of how the test was executed
- **Success Criteria:** Pass/fail conditions, and if they passed or not
- **Result:** The observed outcome in more detail

7.2 Preliminary Tests

Test A1: Video Stream Module

Objective

Verify that the video streaming subsystem correctly captures frames from the camera and transmits them to remote client via WebSocket.

Related Requirements

- **FR-02 (partly)**: Stream live video from rover camera to operator interface

Procedure

1. Run the server handling video streaming on the JetBot
2. From a separate machine on the same network, connect a WebSocket client to the server
3. Verify that frames are received and display live video
4. Test disconnect and reconnection handling

Success Criteria

Criteria	Results
Server Starts without error	Pass
Client receives and displays live video	Pass
Server handles disconnection and reconnection gracefully	Pass

Table 11: Test A1 success criteria

Result

All sucess criteria were met. The video streaming module successfully transmitted live video frames to the remote client. The server handled both initial connection and reconnection scenarios without errors.

Test A2: Motor Control and Calibration

Objective

Verify that motor control module correctly drives the rover upon receiving direct movement commands over HTTP. Simultaneously measure baseline speed and RPM (rotations per minute) measurements for movement calibration.

Related Requirements

FR-01 (partly): Control rover movement in multiple directions

Procedure

1. Place the rover on a flat surface
2. Send a movement command directly over HTTP for every possible direction
3. Position the rover on a marked location
4. Send a forward movement command for exactly 10 seconds
5. Measure the total distance traveled
6. Calculate the motor RPM based on wheel circumference and distance

Success Criteria

Criteria	Results
Rover moves the correct direction for all possible commands	Pass
Movement is reasonably consistent and straight during measurement	Pass
Distance can be reliably measured	Pass

Table 12: Test A2 success criteria

Result

The rover successfully moved in the correct direction for every possible movement command given directly over HTTP The rover traveled 274 cm in 10 seconds, resulting in a calculated motor speed of 41 RPM. This baseline measurement can later be used for movement calibration.

Test A3: Backend-to-Jetbot Integration

Objective

Verify that the Raspberry Pi Backend can successfully communicate with the JetBot via HTTP to send movement commands.

Related Requirements

- **FR-01 (partly):** Control rover movement
- **NFR-01 (partly):** Operate on portable, battery-powered platform (validates deployment architecture)

Procedure

1. Start Docker container for the Raspberry Pi backend
2. Trigger movement commands from the backend
3. Verify the JetBot receives the commands and moves accordingly

Success Criteria

Criteria	Results
Docker container starts without errors	Pass
Backend successfully connects to JetBot API	Pass
Movement commands are received by JetBot	Pass
Rover responds with correct physical movement	Pass

Table 13: Test A3 success criteria

Result

The backend successfully communicated with the JetBot API. Movement commands sent from the Raspberry Pi were correctly received and executed by the rover.

7.3 System Tests

Test B1: End-to-End Video Streaming

Objective

Verify that the complete video streaming pipeline function correctly from camera through to control interface. In the process, also verify that the control interface can in fact be accessed

Related Requirements

- **FR-02:** Stream live video from rover camera to operator interface
- **FR-03**: Provide remote access to control interface
- **NFR-02**: Control interface accessible using standard web browsers without installation
- **NFR-03:** Video feed maintains minimum 20 FPS

Procedure

1. Start the server on the JetBot, and the backend server on the Raspberry Pi
2. Open the web interface in Firefox on a standard laptop
3. Verify live video feed displays in the browser
4. Verify that target frame rate is met

Success Criteria

Criteria	Results
JetBot camera stream initialises	Pass
RPi backend receives and forwards frames	Pass
Frontend displays live video feed	Pass
Frame rate is acceptable for operation	Pass

Table 14: Test B1 success criteria

Result

The complete video pipeline functioned correctly, validating remote video streaming (FR-02) and remote access capabilities (FR-03), we the control interface could be accessed. The web interface loaded in Firefox without requiring any software installation (NFR-02) and the operator was able to view a continuous live video feed. Inspection indicated smooth playback achieving the target frame rate of a 20 FPS minimum (NFR-03).

Test B2: Integrated Disaster Response Mission Simulation

Objective

Verify that the complete integrated rover system can perform a full search and rescue mission, including remove navigation through obstacles and human detection at multiple locations.

Related Requirements

- **FR-01:** Control rover movement in multiple directions
- **FR-02:** Stream live video from rover camera to operator interface
- **FR-03:** Provide remote access to control interface
- **FR-04:** Identify humans and visually indicate them on video feed
- **FR-05:** Support toggling automatic human detection
- **FR-06:** Display rover's estimated position on map (if position tracking enabled)
- **FR-08:** Display current rover operational status
- **FR-09:** Core functionality operates if optional features fail
- **FR-10:** Allow operator to lock movement controls
- **FR-11/FR-12:** Calculate and visualize navigation paths (if pathfinding enabled)
- **FR-13:** Support autonomous navigation (optional test component)
- **NFR-02:** Control interface accessible via standard web browser
- **NFR-03:** Video feed maintains minimum 20 FPS (during manual navigation)

Test Enviorment

The test was completed in an obstacle course simulating a disaster environment.

The obstacle course consisted of the following elements:

- **Start position:** Clear area where the rover starts the test.
- **Obstacles:** -
- **Search Zones:** 3 designated areas where the rover must stop for detecting
  - Zone 1: Person standing in clear view
  - Zone 2: Person sitting (seemingly stuck) behind/partially under table or chair
  - Zone 3: Person lying partially obscured by table or chair

Procedure

Phase 1: System Initialization

1. Power on JetBot and Raspberry pi (BE and FE)
2. Operator opens control interface from laptop
3. Verify that the connection has been successfully established, and that video streaming and interface functions correctly.

Phase 2: Manual Navigation Mission

4. **Obstacle Navigation:** Operator manually controls rover from Start position through obstacle course using only live video feed (they must *not* look at the psychical rover). This step also repeats between different search zones:
   - Navigate around chairs
   - Pass through narrow passage
   - Navigate around tables and scattered "debris"
   - Maintain controlled movement without significant collisions

- Note any potential issues with latency, frame rate, control or other.

8. **Reposition survivor:** Have the survivor in an obscurred position while the rover continuers
9. **Search area - Survivor Lying Down**
   - Position rover at zone 2 with the survivor in view.
   - Lock controls and enable human detection
   - Wait a few seconds for the detection to process and verify detection of partially obscured person
   - Disable detection, unlock controls

Phase 3 - Path Visualization

10. With the rover at zone 3, set the destination to the endpoint of the course, and observe if path visualization on interface shows current route from current position to the end.
11. Manually navigate rover to End position following displayed path (if available)

Phase 4 (extra) - Autonomous Navigation

12. Determine if autonomous navigation is functionale
    - If yes: Configure system with return path (End → Start), and continue this phase
    - If no: End test
13. Initiate autonomous navigation and observe rover follow premade grid path
14. Monitor the rover as well as the position tracker on the map interface.
15. Verify rover successfully reaches start position or note where autonomous control fails

Success Criteria

ID	Criteria	Results
Section A - Initialization		
A1	Web interface loads without requiring software installation	Pass
A2	Video feed display sucessfully	Pass
A3	Rover status indicators display correctly	Pass
Section B - Manual Navigation & Detection		
B1	Operator successfully navigates full obstacle course using video feed only	Pass
B2	Video stream maintains smooth playback (20+ fps) during navigation	Pass
B3	Rover movement successfully prevents movement when enabled	Pass
B4	Human detection toggle functions (both enable/disable)	Pass
B5	Human detected in Zone 1 (clear view) with detection box displayed	Pass
B6	Human detected in Zone 2 (partially obscured)	Pass (barely)
B7	Human detected in Zone 3 (lying down)	Pass (barely)
Section C - Path Visualization		
C1	Path visualization displays on map interface	Pass
C2	Path updates when rover position changes	Pass
Section D - Autonomous Navigation		
D1	Rover follows path without manual input	Pass
D2	Position tracking shows updated location during autonomous movement	Pass
D3	Rover successfully completes entire autonomous return journey	untested

Table 15: Test B2 success criteria

Result

The computer vision system had frequent issues with detecting humans, likely due to the model size being very small, along with the CNN used in our particular case being of the YOLO-nano. Additionally limited computational power lead us to have around 1fps with CV turned on, of course leading to less frames analysed, and thereby the likelyhood of a detected human decreasing significantly

the autonemus driving worked as well as it could since the driving was unreliable so on the software part it was perfect it sent the correct comands and followed the path as it should but hardware part the exequtions was inadequate and was of by 50 - 75 cm from the desination on the testing ground (4.2m * 3.8m)

8 Discussion

8.1 Jetbot setup

During the setup process of the jetbot several challenges where encountered, of which i'd like to take the time to discuss the most severe ones here;

ResizeFS

Nvidia had included a binary under `/usr/lib/NVIDIA/resizefs` that would force the sdcard to be 22GB, and overwrite any other size provided. this meant it was completely impossible to run updates, install custom software, or even transfer files to it. whilst it's suspected that this is due to nvidia wanting to force the use of the jupyter environment due to their incompatible jetson-io lib, that i'll take about in a bit, it was detremental for us, and took quiet a while to discover as this was completely undocumented.

After detection and subsequent deletion of the troublesome binary, the partition could be expanded using `fdisk /dev/sda` and thereafter grown using `resize2fs`

Linux distribution

The linux distribution installed is a depricated ubuntu 18.04. This is an 8 year outdated version running python 2. After a bunch of tinkering we managed to get 3.8 on it, but this was still far below the versions our software had been tailored to, and major rewrites where therefore necesarry. Additionally a dedicated backend server (the raspberry pi) was necessarry as it simply wasn't feasible to get the entire backend and frontend running on the jetbot itself, as a critical dependency, Pytorch, did not support arm64 for python 3.8

Jetson-IO

Jetson-IO is a library the nvidia jetson uses to communicate with extension boards, such as the jetbot, via gpio. The only problem is that jetson-io is dependent on a much newer version of python than we were able to get running on ubuntu, again likely due to nvidia wanting to force jupyter use. Several attempts where made to patch the library, but unfortunately all ended up unsuccessfull, as the IO requests must go through some proprietary controllers, that ensure that the library have not been modified

This forced us to use jupyter for our movement firmware, and thereby forced the jetbot firmware to be segmented into a locally running camera firmware, and a cloud based movement firmware that requires a network connection

Jupyter websockets

Due to the broken Jetson-IO lib combined with the hopelessly depricated python version, we were eventually forced to use Jupyter to cut the time losses. this however meant that we would be sacrificing websockets for our movement firmware, as jupyters own control flow cannot be interupted nor overwritten, and as a result websockets are physically impossible to implement. The implementation therefore ended on http requests.

Here a consideration was made wether all the firmware for the jetbot should be run via Jupyter, but this idea was scrapped as streaming a live video feed over pure http get and post requests really isn't an ideal solution. This resulted in the affore mentioned segmentation of the jetbot firmware, and communication protocols

Controllability

The Controllability of the jetbot was... unideal

The movement scheme is very basic as the motors lack any sort of feedback, and therefore the jetbot doesn't know how far the motors moved. This leads to speed drifting slowly over time due to floating point errors, and leads to the jetbot, even at the lowest speeds, not being able to drive in a straight line.

Additionally the jetbot acceleration scheme is too aggressive for the limited amount of grip the wheels contain, leading to the bot "jumping" several cm to the left or right upon acceleration

The solution to this was to carefully tune our calibration and individual motor values to get it as close to straight as possible

A solution involving an IMU was also proposed, unfortunately the provided IMU (bmi160) seemed to have had an issue (or more likely miswirering from our part) resulting in random noise that made the data unreliable and we were therefore unable to use this for a reliable estimation of the rovers position as a result our movement scheme became as rudementary as it did. Our movement scheme has also become extremely fragile because of this, as floating point errors will naturally cause it to drift, and if it ever where to be slowed down unexpectedly, we would have no way to detect this, and we would assume the robot was several feet ahead of it's actual position

This behaviour was also mentionned in several discussions with other groups, indicating it's not our particular unit there's defective, but rather a design flaw from nvidia's side, not having designed the jetbot for any sort of precision.

8.2 Computer vision

Computer vision, just like any other AI based system, comes with it's limitations, stemming from dataset, input size, computational power and training length, among many others This have lead to some unique challenges, such as testing it in the first place, since it would only detect people far away, as this is what it was trained on, in the folowing section thise chalenges will be discussed

Potential challenges

Computer vision is an AI based system, which means that we're only able to recognise what was in the dataset. This has brought upon several concerns about people of different skin colour, or different religious believes (and hence clothing). Some examples can be found below

since the dataset dose contain people of color we can only hypothesize that it shouldn´t have problems, but its theaoreticly posible since the ai migth be taken colors in to consideration and the majority of the data set is comprised of whight males this is sadly a limitation of most datasets. due to the nature of our group we cant say for sure only that it dosen´t have problems on prople of european and asian desent

In the case of body covering clothing that alters your slhouette, such as costumes or certain capes, the person in question would very likely not be detected, especially if it covers any remotely human looking qualities as well. The cv system would more than likely just view the person in question as an object, and move on, although this is purely hypothetical, and have not been tested

8.3 IMU

As mentioned before, we attempted to create a reliable possitioning estimation based on data gathered from a BMI160 IMU communicating woith the jetbot over I2C. this however had several issues

For a starter the jetbot only exposes a single I2C interface, and has it hardwired so you can't use any other gpio pins for this purpose. The solution was to create Y-splitter gpio cables, to have both the IMU and the JETBOT daughter board interface with the jetson at the same time. This is luckily possible as I2C is essentially a merge between SPI and UART. Like UART it only uses 2 wires, but like SPI it support multiple slaves on one connection, as it can use address resolution to tell theese apart. Jetbot lived on 0x68, so we simply set the IMU to 0x69

The much greater issue was unreliable data readouts. No matter how many times we ran through calibration, or how we chose to read or process the data, the results where the same. Data wouldn't stay consistent, and we eventually deemed it unviable for positioning estimation. Now this is very likely due to a missing pulldown resistor somewhere on the board, as it looked like electrical noice, the documentation however was lackluster and we were therefore unable to determine on what pin, and of which value the beforementioned resistor should be attached to.

9 Conclusion

The project saught to answer the question proposed in the problem statement of "how we can design a remotely controlled, recon rover suitable for assisting first responders in gathering intelligence & detecting human survivors. Our implemented system, The AURORA, manages to demonstrate that such system is entirely feasible, even under heavy hardware and computationally based constraints

The majority of the user stories where capable of being met, along with all of our must and should haves, and many of the could haves.

The computer vision system proofed in our benchmarking to be outstanding in reliability scoring an F1 score of 0.91 out of 1, and only the computational limitations are left to blame for the downsizing of the model, and low frame rate that eventually lead to the problems descirbed above.

The A* and autonomous driving algorithm equally so proofed to function perfectly, and correctly choose the optimal path, and send the desired commands to the rover at every step along the way. If it had more relible positional feedback, there would've likely been no issues observed in the navigational parts of the project.

Additionally the solution with a web based frontend, allowing any device without prior configuration to interface with the rover remotely worked brilliantly, and without any hickups.

In conclusion; The software part of the system functioned near perfectly, and the project was only kept back by signifficant hardware constraints. Even with theese constraints however it still serves as a usable Proof of Concept, and it is our believe that with greater hardware and computational power, our system would function exactly as intended

Images

Figure 1 Kanban board workflow showing task organization ... 8
Figure 2 finised astar search visulisation ... 14
Figure 3 finised dijksra search visulisation ... 14
Figure 4 usecase diagram ... 19
Figure 5 flowchart of the deployment process ... 20
Figure 6 Frontend message routing flowchart ... 28
Figure 7 Plot of $F(x) = x \log(x)$ ... 39
Figure 8 precision is predicted positive instances are actually correct ... 44
Figure 9 Recall is actual positive instances were correct ... 45
Figure 10 PR Curve plots precision against recall ... 45
Figure 11 mean of precision and recall ... 46
Figure 12 Benchmark of different model formats [30] ... 47

Tables

Table 1 Emergency Response Team User Stories ... 16
Table 2 Incident Commander User Stories ... 17
Table 3 Use Cases ... 18
Table 4 Must have functional requirements ... 21
Table 5 Should have functional requirements ... 21
Table 6 Could have functional requirements ... 22
Table 7 Must have non-functional requirements ... 23
Table 8 Should have non-functional requirements ... 23
Table 9 Could have non-functional requirements ... 23
Table 10 Testing Enviorment ... 53
Table 11 Test A1 success criteria ... 54
Table 12 Test A2 success criteria ... 55
Table 13 Test A3 success criteria ... 55
Table 14 Test B1 success criteria ... 56
Table 15 Test B2 success criteria ... 59

Kode Blok

Listing 1 WebSockets hook setup (`useRoverWebsockets.ts`) ... 27
Listing 2 WebSockets hook setup (`useRoverWebsockets.ts`) ... 28
Listing 3 sartup of backend websocket code(`main.py`) ... 29
Listing 4 recieving BE websocket (`main.py`) ... 30
Listing 5 sending BE websocket (`main.py`) ... 31
Listing 6 Rover state initialization (`state.py`) ... 33
Listing 7 Some of the setter functions (`state.py`) ... 34
Listing 8 State synchronization (`state.py`) ... 35
Listing 9 Observer registration (`main.py`) ... 35
Listing 10 node prameters (`a_star.py`) ... 36
Listing 11 checks and helper funktions (`a_star.py`) ... 36
Listing 12 traces the path that A* finds (`a_star.py`) ... 37

Listing 13 the a* algorithm code(`a_star.py`) 38
Listing 14 position tracker code(`position_tracker.py`) 40
Listing 15 autonumus driving helper fuctions (`state.py`) 41
Listing 16 autonumus driving fuctions (`state.py`) 42
Listing 17 Camera Initialization (`visuals_handler.py`) 48
Listing 18 Camera Initialization (`visuals_handler.py`) 49
Listing 19 Websocket exposer for jetbot camera feed (`CameraFeedExposer.py`) 51
Listing 20 Jetbot movement firmware implemented in jupyter notebook 52

Bibliography

[1] U.S. Bureau of Labor Statistics, "Fatal Occupational Injuries to Emergency Responders." [Online]. Available: https://www.bls.gov/iif/factsheets/archive/fatal-occupational-injuries-emergency-responders-2011-15.html

[2] National Science Foundation, "Robot-Assisted Urban Search and Rescue from 9/11 to Now." [Online]. Available: https://www.nsf.gov/events/robot-assisted-urban-search-rescue-911-now-where

[3] National Science Foundation, "Robots to the rescue." [Online]. Available: https://www.nsf.gov/news/robots-rescue-0

[4] [Online]. Available: https://apps.usfa.fema.gov/firefighter-fatalities/

[5] A. Sawaya, A. Regina, and R. Menezes, "Hydrogen Sulfide Toxicity," in *StatPearls*, StatPearls Publishing, 2023. [Online]. Available: https://www.ncbi.nlm.nih.gov/books/NBK559264/

[6] R. Varma, "The Bhopal Disaster of 1984," *Bulletin of Science, Technology & Society*, vol. 25, no. 1, pp. 37–45, 2005, doi: 10.1177/0270467604273822.

[7] "Structural Collapse Guide." [Online]. Available: https://www.osha.gov/emergency-preparedness/guides/structural-collapse

[8] Y.-J. Shang, "3-D computer vision," [Online]. Available: https://link.springer.com/content/pdf/10.1007/978-981-19-7580-6.pdf

[9] A. Paszke *et al.*, "Automatic differentiation in PyTorch," 2017.

[10] [Online]. Available: https://landscape.pytorch.org/?item=modeling%E2%80%93language%E2%80%93flair

[11] google, "Tensorflow," [Online]. Available: https://www.tensorflow.org/

[12] C. rand, "TensorFlow Performance Analysis," [Online]. Available: https://towardsdatascience.com/tensorflow-performance-analysis-314b56dceb59/

[13] GeeksForGeeks, "Differences between Pytorch and Tensorflow," [Online]. Available: https://www.geeksforgeeks.org/python/difference-between-pytorch-and-tensorflow/

[14] M. Moses and K. P. Extension, "Robotics in Disaster Response: Enhancing Search and Rescue Operations," vol. 3, pp. 51–56, 2024.

[15] H. Surmann *et al.*, "Lessons from robot-assisted disaster response deployments by the German Rescue Robotics Center task force," *Journal of Field Robotics*, vol. 41, no. 3, pp. 782–797, 2024, doi: https://doi.org/10.1002/rob.22275.

[16] [Online]. Available: https://jetbot.org/master/index.html

[17] C. Gregg, [Online]. Available: https://web.stanford.edu/class/archive/cs/cs106b/cs106b.1262/lectures/27-dijkstra/

[18] cp-algorithms, [Online]. Available: https://cp-algorithms.com/graph/dijkstra.html

[19] E. W. Dijkstra, "A Note on Two Problems in Connexion with Graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959, doi: 10.1007/BF01386390.

[20] Y. Yang and Q. Cai, "Research on the A-star Algorithm Based on Path Finding," in *2023 IEEE 3rd International Conference on Data Science and Computer Application (ICDSCA)*, 2023, pp. 199–202. [Online]. Available: https://ieeexplore.ieee.org/document/10392293

[21] O. Geil, "The Mathematical Foundation of A*," *The Mathematical Foundation of A**, 1 BC, [Online]. Available: https://people.math.aau.dk/~olav/undervisning/sp07/astarcor.pdf

[22] [Online]. Available: https://emergencyreports.com/5524-cox-rd-east-milton-fl-32583-d9dgg07p7m6x

[23] C. Fu, R. Duan, D. Kircali, and E. Kayacan, "Onboard Robust Visual Tracking for UAVs Using a Reliable Global-Local Object Model," *Sensors*, vol. 16, no. 9, p. 1406, Aug. 2016, doi: 10.3390/s16091406.

[24] WebSocket.org, "The Road to WebSockets." Accessed: May 21, 2024. [Online]. Available: https://websocket.org/guides/road-to-websockets/

[25] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, "Design Patterns: Elements of Reusable Object-Oriented Software," Addison-Wesley Professional, 1994, ch. 3.

[26] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, "Design Patterns: Elements of Reusable Object-Oriented Software," Addison-Wesley Professional, 1994, ch. 5.

[27] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788. doi: 10.1109/CVPR.2016.91.

[28] Ultralytics, "Home - Ultralytics YOLO Docs." [Online]. Available: https://docs.ultralytics.com/

[29] N. Jegham, C. Y. Koh, M. Abdelatti, and A. Hendawi, "YOLO Evolution: A Comprehensive Benchmark and Architectural Review of YOLOv12, YOLO11, and Their Previous Versions." [Online]. Available: https://arxiv.org/abs/2411.00201

[30] Ultralytics, "YOLO12: Attention-Centric Object Detection." [Online]. Available: https://docs.ultralytics.com/models/yolo12/

[31] Y. Zdemir, "AiTV Dataset: Human Detection at Disasters." [Online]. Available: https://www.kaggle.com/datasets/yusufzdemir23/aitv-dataset-human-detection-at-disasters

[32] R. A. Nihal, B. Yen, K. Itoyama, and K. Nakadai, "C2A Dataset: Human Detection in Disaster Scenarios." [Online]. Available: https://www.kaggle.com/datasets/rgbnihal/c2a-dataset/data

[33] R. Padilla, S. Netto, and E. da Silva, "A Survey on Performance Metrics for Object-Detection Algorithms," 2020, p. . doi: 10.1109/IWSSIP48289.2020.

[34] Ultralytics, "Quick Start Guide: Raspberry Pi with Ultralytics YOLO11." Accessed: Dec. 18, 2025. [Online]. Available: https://docs.ultralytics.com/guides/raspberry-pi

[35] nvidia, [Online]. Available: https://developer.nvidia.com/embedded/jetson-nano

[36] S. K. amd David Rollinson and H. Choset, "Modularity for Maximum Mobility and Manipulation: Control of a Reconfigurable Legged Robot with Series-Elastic Actuators." [Online]. Available: https://ieeexplore.ieee.org/document/7442943

[37] S. Yang, L. Wang, Y. Huang, and H. Chen, "A Safe and Efficient Self-Evolving Algorithm for Decision-Making and Control of Autonomous Driving Systems," *IEEE Transactions on Intelligent Transportation Systems*, vol. 26, no. 8, pp. 11466–11478, 2025, doi: 10.1109/TITS.2025.3576372.

[38] EM-DAT, CRED / UCLouvain and Our World in Data, "Global damage costs from natural disasters." 2024.

[39] Meta and the React community, "React API Reference." [Online]. Available: https://react.dev/reference/react

[40] Python Wiki contributors, "GUI Programming in Python." Accessed: May 21, 2024. [Online]. Available: https://wiki.python.org/moin/GuiProgramming

[41] Raspberry Pi Foundation, "Remote access - Raspberry Pi Documentation." Accessed: Dec. 15, 2025. [Online]. Available: https://www.raspberrypi.com/documentation/computers/remote-access.html

[42] A. Murali, P. Raj, H. Subramanian J, and P. R. Chelliah, in *Architectural Patterns*, O'Reilly Media, 2018, ch. 2. [Online]. Available: https://www.oreilly.com/library/view/architectural-patterns/9781787287495/e05796aa-bd3e-4607-b5ec-fddee6e0d42a.xhtml

[43] G. Jocher and J. Qiu, "Ultralytics YOLO11." [Online]. Available: https://github.com/ultralytics/ultralytics

[44] Python Software Foundation, "tkinter — Python interface to Tcl/Tk." [Online]. Available: https://docs.python.org/3/library/tkinter.html

[45] Riverbank Computing, "PyQt - Introduction." [Online]. Available: https://riverbankcomputing.com/software/pyqt/intro

10 Appendix

link to the videos of our tests: https://files.ox.voxvoltera.com/s/Zqd6XZJdYYFn6PE